

FESTARA

Collaborative Event Planning and Expense Management



Anas Altaf (MC-331)

Hammad Ansar (MC-304)

Muhammad Sami Ullah (MC-336)

Supervised By

Tahira Iqbal

Submitted for the partial fulfillment of BS Computer Science degree to the Faculty of
Engineering & CS

**DEPARTMENT OF COMPUTER SCIENCE
NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD**

October, 2026

ABSTRACT

Families, friend groups, and university societies regularly plan events together, yet most of them still coordinate through a mix of WhatsApp messages, spreadsheets, and memory. Guest lists live on one person's phone, expenses are mentioned in chat and then forgotten, and tasks are handed out verbally with no record of who agreed to do what. Existing tools each solve a slice of this problem: Eventbrite handles public ticketing, Splitwise settles shared debts after the fact, and professional wedding planners such as Aisle Planner are priced for businesses rather than households. No affordable tool brings event planning, shared expense tracking, and task management together for small private groups. Festara is a web application built to close that gap for groups of roughly 3 to 50 people.

Festara lets one person create an event, set a budget, and invite others through a shareable link with a specific role: Admin, Member, or Guest. The system is divided into eight modules: User Authentication, Event Management, Role-Based Access Control, Guest List and RSVP, Expense and Budget Tracking, Tasks and Arrangements, an Admin Dashboard, and an AI Features module planned for the second phase. The project follows the Scrum process model in two-week sprints. It is built with Next.js 16 and Tailwind CSS on the front end, Supabase (PostgreSQL, Auth, and Realtime) as the backend, and Vercel for hosting. Access rules are enforced inside the database through PostgreSQL Row Level Security, so a user can only read or change the rows that their role permits, regardless of what the client sends.

This report documents the project at the 40 percent milestone. The requirements, use cases, and complete system design for all modules are presented, together with the implementation of the three foundation modules: authentication, event management, and role-based access control with invite links. These modules are being validated through black-box functional testing, integration testing of the invite flow, and direct security tests against the database policies, using the test cases defined in this report. The remaining modules, including expense tracking with an 80 percent budget warning and the AI checklist and budget estimator, are scheduled in the project plan. Known limitations include the lack of in-app payments, the need for an internet connection, and link-only invitations in the first release.

CERTIFICATE

Dated: _____

Final Approval

It is certified that the project report titled "Festara: Collaborative Event Planning and Expense Management" submitted by Anas Altaf, Hammad Ansar, and Muhammad Sami Ullah for the partial fulfillment of the requirement of "Bachelors Degree in Computer Science" is approved.

COMMITTEE

Dr. Abdul Majid Somroo
HoD Computer Science

Signature: _____

Syed Zain ul Abdin
Program Coordinator CS

Signature: _____

Tahira Iqbal
FYP Coordinator CS

Signature: _____

Tahira Iqbal
Supervisor

Signature: _____

DECLARATION

We hereby declare that our dissertation is entirely our work and genuine/original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Group Members

Signature

1. Anas Altaf (MC-331)

2. Hammad Ansar (MC-304)

3. Muhammad Sami Ullah (MC-336)

PLAGIARISM CERTIFICATE

This is to certify that the project entitled "Festara: Collaborative Event Planning and Expense Management" is being submitted here for the award of the "Degree of Bachelor" in "Computer Science". This is the result of the original work by Anas Altaf, Hammad Ansar, and Muhammad Sami Ullah under my supervision and guidance. The work embodied in this project has not been done earlier for the basis of the award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of my knowledge and belief.

Turnitin Originality Report	Processed on: _____ ID: _____
Word Count	_____
Similarity Index	_____ %
Similarity by Source	Internet Sources: _ % Publications: _ % Student Papers: ____ %

Date: _____

Tahira Iqbal
(Supervisor)

TURNITIN ORIGINALITY REPORT

**"Festara: Collaborative Event Planning and Expense Management" by Anas Altaf,
Hammad Ansar, and Muhammad Sami Ullah**

From Tahira Iqbal

Processed on	_____
ID	_____
Word Count	_____
Similarity Index	_____ %
Internet Sources	_____ %
Publications	_____ %
Student Papers	_____ %

SOURCES: (Attach the source list from the Turnitin report here.)

ACKNOWLEDGMENT

We are grateful to Allah Almighty for giving us the ability to carry out this work. We thank our supervisor, Tahira Iqbal, for her guidance, her patience during our meetings, and the practical feedback that shaped Festara's scope and design. We also thank the faculty of the Department of Computer Science, National University of Modern Languages, Islamabad, for the knowledge and support they provided throughout our degree. Finally, we thank our families and friends, whose own weddings, trips, and society events gave us the problem this project tries to solve.

TABLE OF CONTENTS

Chapter 1: INTRODUCTION	1
1.1 Introduction	2
1.2 Motivation	2
1.3 Problem Statement	3
1.4 Goals and Objectives	3
1.5 Scope of the Study	4
1.6 Process Model	4
1.7 Nature of the Project: A Cloud-Based Collaborative Web Application	5
1.8 Overview / Organization of the Report	5
Chapter 2: BACKGROUND AND EXISTING WORK	6
2.1 Introduction	7
2.2 Important Constructs of the Application Domain	7
2.2.1 Collaborative Event Planning	7
2.2.2 Shared Expense and Budget Management	7
2.2.3 Role-Based Access Control	7
2.2.4 Backend-as-a-Service and Row Level Security	8
2.3 Existing Studies / Systems	8
2.3.1 Related Literature Review	8
2.3.2 Related Systems / Applications	9
2.4 Comparison of Existing Systems	10
2.5 Summary	11
Chapter 3: REQUIREMENTS SPECIFICATION	12
3.1 Introduction	13
3.2 Interface Requirements	13
3.2.1 Hardware Interface Requirements	13
3.2.2 Software Interface Requirements	13
3.3 Functional Requirements	14
3.4 Use Case Model	16
3.5 Use Cases	17
3.5.1 Use Case 1: Create Event	17
3.5.2 Use Case 2: Join Event via Invite Link	18
3.5.3 Use Case 3: Manage Member Roles	19
3.5.4 Use Case 4: Log Expense	19

3.5.5 Use Case 5: Confirm RSVP	20
3.6 Non-Functional Requirements	21
3.6.1 Performance	21
3.6.2 Reliability	21
3.6.3 Security	21
3.6.4 Usability	22
3.6.5 Consistency	22
3.6.6 Portability	22
3.7 Resource Requirements	22
3.7.1 Equipment	22
3.7.2 Funds	23
3.7.3 Human Effort	23
3.8 Database Requirements	24
3.9 Project Feasibility	25
3.9.1 Technical Feasibility	25
3.9.2 Operational Feasibility	25
3.9.3 Economic Feasibility	25
3.9.4 Legal and Ethical Feasibility	25
3.10 Summary	25
Chapter 4: SYSTEM MODELLING	27
4.1 Introduction	28
4.2 System Design	28
4.3 Design Approach	29
4.4 Interface Design	29
4.4.1 High Fidelity Prototype	29
4.5 Data Flow Diagrams	30
4.6 4+1 View Model of Architecture	32
4.6.1 Logical View	32
4.6.2 Process View	33
4.6.3 Development View	35
4.6.4 Physical View	36
4.7 Entity Relationship Diagram	37
4.8 Summary	38
Chapter 5: IMPLEMENTATION	39
5.1 Introduction	40

5.2 Development Environment and Project Structure	40
5.3 Modules of the FYP	40
5.3.1 User Authentication Module	40
5.3.2 Event Management Module	41
5.3.3 Role-Based Access Control Module	42
5.3.4 Planned Implementation of Remaining Modules	44
5.4 H/W Module Details	45
5.5 Summary	45
Chapter 6: RESULT / TESTING, ANALYSIS AND VALIDATION	46
6.1 Introduction	47
6.2 Testing Strategy	47
6.2.1 Testing Environment	47
6.2.2 Black Box Testing	47
6.2.3 Unit Testing	47
6.2.4 Integration Testing	47
6.2.5 Security Testing of Access Policies	48
6.3 Test Cases	48
6.3.1 Test Case TC-07: Join Event via Valid Invite Link	48
6.4 Validation of Non-Functional Requirements	50
6.5 Summary	50
Chapter 7: CONCLUSION AND FUTURE WORK	52
7.1 Introduction	53
7.2 Progress Review against Objectives	53
7.3 Remaining Work and Timeline	53
7.4 Limitations	54
7.5 Future Work	54
7.6 Summary	55
APPENDIX I: DATA DICTIONARY	56
profiles	56
events	56
event_members	57
invitations	57
guests	57
expenses	58
tasks	58

activity_log	58
APPENDIX II: USER MANUAL (COMPLETED MODULES)	59
Registering and Logging In	59
Creating an Event	59
Inviting People	59
REFERENCES	60

LIST OF FIGURES

Figure 1.1: Scrum process model adopted for Festara	5
Figure 3.1: Use case diagram of Festara	17
Figure 4.1: System architecture of Festara	28
Figure 4.2: Prototype of the login and registration screen	30
Figure 4.3: Prototype of the home dashboard	30
Figure 4.4: Prototype of the members and invite screen	30
Figure 4.5: DFD level 0 (context diagram) of Festara	31
Figure 4.6: DFD level 1 of Festara	32
Figure 4.7: Class diagram of Festara	33
Figure 4.8: Activity diagram for creating an event and joining through an invite link	34
Figure 4.9: State diagram of a task	35
Figure 4.10: Sequence diagram for accepting an invitation	35
Figure 4.11: Component diagram of Festara	36
Figure 4.12: Deployment diagram of Festara	37
Figure 4.13: Entity relationship diagram of Festara	37
Figure 5.1: Implemented login screen	41
Figure 5.2: Implemented create-event form	42
Figure 5.3: Role helper function and Row Level Security policies on the events table	43
Figure 5.4: Server action for accepting an invitation	43
Figure 5.5: Implemented members and invite screen	44
Figure 7.1: Project timeline showing completed and planned work	54

LIST OF TABLES

Table 2.1: Summary of Reviewed Literature	9
Table 2.2: Summary of Existing Systems	10
Table 2.3: Feature Comparison of Existing Systems and Festara	11
Table 3.1: Hardware Interface Requirements	13
Table 3.2: Software Interface Requirements	14
Table 3.3: Functional Requirements	14
Table 3.4: Full Dress Use Case for Create Event	17
Table 3.5: Full Dress Use Case for Join Event via Invite Link	18
Table 3.6: Full Dress Use Case for Manage Member Roles	19
Table 3.7: Full Dress Use Case for Log Expense	20
Table 3.8: Full Dress Use Case for Confirm RSVP	20
Table 3.9: Equipment and Tools with Justification	23
Table 3.10: Task Division and Human Effort	24
Table 5.1: Project Folder Structure	40
Table 5.2: Role Permission Matrix	42
Table 6.1: Test Case TC-07 in Full Format	48
Table 6.2: Summary of Test Cases	49
Table 6.3: Non-Functional Requirement Validation Plan	50
Table 7.1: Progress against Project Objectives	53

Chapter 1
INTRODUCTION

1.1 Introduction

People have always organized events together. A wedding pulls in the whole family, a road trip needs everyone to agree on plans and split costs, and a university society event runs on a committee where a dozen people each handle a different job. The difficulty is rarely that people do not know how to plan. It is that they have no shared place to plan in. Coordination happens across chat groups, phone contacts, paper lists, and spreadsheets, and the person in charge ends up holding the whole picture in their head.

Software for events has grown quickly, but it has grown in separate directions. Ticketing platforms such as Eventbrite serve public events with paying attendees [5], [9]. Expense-splitting tools such as Splitwise settle who owes whom once money has already been spent [10]. Full wedding-planning suites cover almost everything, but they are priced and designed for professional planners rather than for a father organizing his daughter's wedding [7]. Industry surveys report that budget control is the most cited pain point in event planning, with 95 percent of planners naming rising costs as a top concern [5], and that 89 percent of buyers of collaboration software rate access control and expense tracking as important or highly important [8].

Festara is a web application that brings event planning, shared expense tracking, and task management into one place for small private groups [1], [2]. A group creates an event, sets a budget, and invites members with defined roles. From that point everyone works from the same screen: guests are added and tracked, expenses are logged by whoever spends the money, tasks are assigned with deadlines, and the Admin sees the overall state of the event on a single dashboard. The application runs in any modern browser, needs no installation, and is free for basic personal use.

1.2 Motivation

The idea for Festara came from the team's own experience. In the months before this project, members of the team took part in family weddings, a northern-areas road trip with friends, and a society event at the university. Each one ran into the same problems. On the trip, five people paid for petrol, food, and rooms at different times, and at the end nobody could reconstruct who had paid for what. At the wedding, the guest list existed in three versions held by three relatives. At the society event, two committee members both assumed the other had booked the sound system.

None of these problems needed advanced technology to solve. They needed a single shared record with clear ownership: one list of guests, one ledger of expenses against one

budget, and one list of tasks with a named person and a date next to each. The motivation for Festara is to give ordinary groups that shared record without a subscription fee or the learning curve of professional tools.

1.3 Problem Statement

Small groups that organize private events, such as families, friend circles, and student societies, lack a single, affordable tool for coordinating their guests, shared spending, and responsibilities. Existing applications cover only part of the work, so groups fall back on chat messages and spreadsheets that have no structure, no role-based control, and no live view of the budget. As a result, expenses go untracked, tasks are missed, guest responses are lost, and the burden of coordination falls on one person. The problem this project addresses is the design and development of a browser-based system in which a group can plan an event collaboratively, with each participant's permissions defined by their role, and with spending and progress visible to the organizer in real time.

1.4 Goals and Objectives

The overall goal of the project is to deliver a working web application that a non-technical group can use to plan a private event from start to finish. The specific objectives are:

1. To provide secure registration, login, password reset, and profile management for users.
2. To allow a user to create, edit, and delete events with a name, type, date, location, description, and total budget.
3. To enforce three roles, Admin, Member, and Guest, so that every action in the system is permitted or denied according to the user's role in that event.
4. To let organizers add guests, share invite links, and track each guest's RSVP status as Pending, Confirmed, or Declined.
5. To let any member log expenses against the event budget by category, and to warn the group when spending crosses 80 percent of the budget.
6. To let organizers create tasks with an assignee and deadline, and to track each task through To Do, In Progress, and Done.
7. To give the Admin a dashboard that summarizes days remaining, members, guests, task completion, budget usage, and recent activity.
8. In the second phase, to use a large language model to generate preparation checklists and suggested budget splits for a given event type and guest count.

1.5 Scope of the Study

Festara is limited to private, invitation-only events for groups of roughly 3 to 50 people. It covers family events such as weddings, engagements, Mehndi, Walima, Eid gatherings, and birthdays; friend-group activities such as road trips, vacations, and outings; university society events such as tech fests, seminars, and annual functions; and small professional use by independent planners who manage client events without custom software.

The following are outside the scope of this project: public ticketing or paid registration, in-app money transfers or payment processing between users, virtual event hosting or video streaming, and enterprise features such as CRM, contracts, or marketing automation. This report covers the work completed up to the 40 percent milestone, which consists of the full requirements and design for every module and the implementation of the User Authentication, Event Management, and Role-Based Access Control modules.

1.6 Process Model

The project follows Scrum, an iterative and incremental framework in which work is delivered in short, fixed-length sprints [16]. Scrum was chosen over the Waterfall model for three reasons. First, the requirements of a social, user-facing product are best refined by showing working screens to users and to the supervisor, and Scrum builds that review into every sprint. Second, the modules of Festara depend on each other in a clear order (authentication before events, events before roles, roles before guests and expenses), which maps naturally onto a sequence of sprints. Third, the team is small, and Scrum's lightweight ceremonies suit three students working alongside their coursework.

Each sprint lasts two weeks. At the start of a sprint the team selects items from the product backlog into a sprint backlog. During the sprint the team holds short daily check-ins over WhatsApp or in person. At the end of the sprint the working increment is deployed to Vercel and demonstrated to the supervisor in the review meeting, and the team holds a short retrospective. Feedback from the review is added back to the product backlog. The cycle is shown in Figure 1.1.

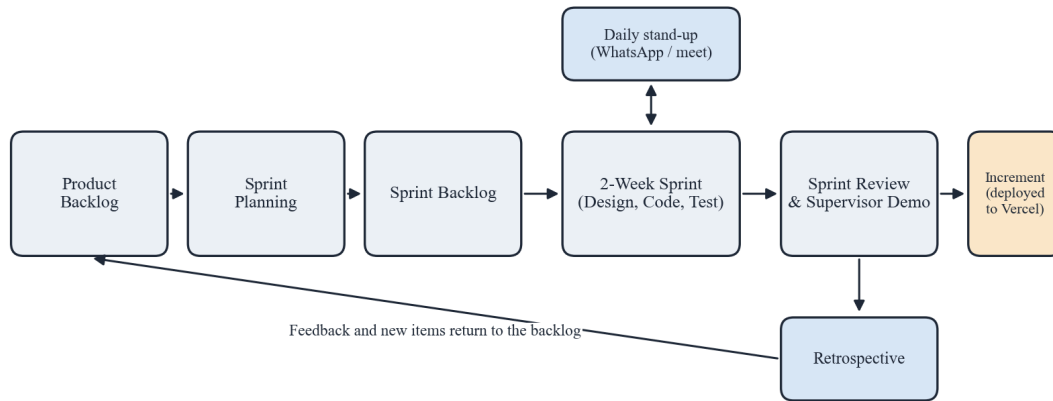


Figure 1.1: Scrum process model adopted for Festara

1.7 Nature of the Project: A Cloud-Based Collaborative Web Application

Festara is a web application built on a serverless, cloud-hosted architecture. The front end is written in Next.js 16 using the App Router with React Server Components, and styled with Tailwind CSS [2]. The backend is provided by Supabase, a Backend-as-a-Service platform that supplies a managed PostgreSQL database, user authentication, and real-time change subscriptions [1]. The application is deployed on Vercel. In the second phase, an AI module will call the OpenAI API through the Vercel AI SDK to generate checklists and budget estimates [3], [4]. The project therefore combines web engineering, database security design, and applied AI.

1.8 Overview / Organization of the Report

The rest of this report is organized as follows. Chapter 2 explains the domain of collaborative event planning, reviews related research and existing applications, and compares them with Festara. Chapter 3 specifies the system's interface, functional, and non-functional requirements, use cases, resource needs, and feasibility. Chapter 4 presents the system model, including the architecture, interface design, data flow diagrams, the 4+1 architectural views, and the entity relationship diagram. Chapter 5 describes the implementation of the modules completed at the 40 percent milestone and the plan for the remaining modules. Chapter 6 defines the testing approach and test cases. Chapter 7 reviews progress against the objectives and outlines the remaining work, limitations, and future directions.

Chapter 2

BACKGROUND AND EXISTING WORK

2.1 Introduction

Festara sits where three areas meet: group event planning, shared expense management, and access control for multi-user applications. This chapter first explains the key concepts of that domain. It then reviews published research on collaborative planning and role-based access control, describes the existing applications that groups use today, and compares them against the features Festara provides.

2.2 Important Constructs of the Application Domain

2.2.1 Collaborative Event Planning

Collaborative event planning is the process by which several people jointly decide and carry out the arrangements for an event: who is invited, where and when it happens, what it costs, and who does each job. Research on group event scheduling shows that the outcome depends heavily on how input from participants is gathered and how decisions are made visible to the group [11]. Planning also changes over time: a guest list grows, a venue is booked, and costs accumulate as the date approaches. A planning tool must therefore support continuous updates from many people rather than a single form filled in once.

2.2.2 Shared Expense and Budget Management

Group events are usually paid for by several people at different times. Shared expense management means recording each payment with its amount, category, payer, and date, so that the group can see total spending against an agreed budget and each member's contribution. Two views matter: a budget view, which compares spending with the planned total, and a contribution view, which shows who has paid for what. Consumer research shows growing demand for budgeting tools that give this kind of visibility [6].

2.2.3 Role-Based Access Control

Role-Based Access Control (RBAC) is an access control model in which permissions are attached to roles and users are assigned to roles, rather than permissions being granted to each user directly [14]. The NIST RBAC standard formalizes this into core RBAC (users, roles, permissions, and sessions), role hierarchies, and separation-of-duty constraints [15]. In Festara, roles are scoped to a single event: the same person can be the Admin of their own wedding and a Member of a friend's trip. This per-event assignment is a direct application of core RBAC, with the event acting as the context in which a role applies.

2.2.4 Backend-as-a-Service and Row Level Security

Backend-as-a-Service (BaaS) platforms provide ready-made database, authentication, and storage services so that developers can focus on application logic. Supabase is an open-source BaaS built on PostgreSQL [1]. A key feature it exposes is Row Level Security (RLS), a PostgreSQL mechanism that attaches policies to tables so that each query only returns or modifies the rows the current user is allowed to access [17]. Because the policy runs inside the database, a user cannot bypass it by calling the API directly. Festara relies on RLS to enforce its role rules.

2.3 Existing Studies / Systems

2.3.1 Related Literature Review

OutWithFriendz (Zhang et al.) [11]. This study built a mobile application that lets a host invite friends, suggest times and venues, and let the group vote. The authors deployed it on iOS and Android and analyzed more than 500 users and 300 real group events. They found that participants' mobility patterns, individual preferences, the host's own preferences, and the dynamics of voting all influence the final decision. The work shows that structured group input improves planning, but it focuses only on choosing a time and place. It does not handle budgets, expenses, guest RSVPs, or task assignment after the decision is made.

Cobi (Kim et al.) [12]. Cobi is a community-informed scheduling tool used to plan the CHI 2013 conference. It collected preferences, constraints, and affinity data from authors through community-sourcing applications, then combined that data with constraint solving in a visual interface for organizers. In the live deployment, organizers considered input from 645 authors and resolved 168 scheduling conflicts. Cobi demonstrates the value of giving organizers one shared view built from many contributors' input. However, it is designed for a large academic conference with professional organizers, not for small private groups, and it does not address money or individual task ownership.

Takeplace (Škrabálek et al.) [13]. This case study describes the analysis, design, and implementation of Takeplace, a web-based service for planning, managing, and promoting professional and academic events. The authors discuss how Web 2.0 technologies allow rich internet applications for event management and online professional communities. The system shows that a browser-based platform can serve as the single workspace for an event's organizing team. Its focus, however, is on public professional events, registration, and promotion, rather than private family or friend events with shared spending.

RBAC models (Sandhu et al.) [14] and the NIST RBAC standard (Ferraiolo et al.) [15]. Sandhu and colleagues defined a family of RBAC models that separates users, roles, and permissions and introduces role hierarchies and constraints. Ferraiolo and colleagues later proposed a unified NIST standard built on that work, defining core RBAC, hierarchical RBAC, and separation-of-duty relations. These papers provide the theoretical basis for Festara's Admin, Member, and Guest roles. Their limitation for this project is that they describe enterprise-wide roles; they do not directly address roles that are scoped to an individual shared object, such as one event, which Festara must support. Table 2.1 summarizes the reviewed literature.

Table 2.1: Summary of Reviewed Literature

Year	Authors	Contribution	Techniques	Limitations
2018	Zhang et al. [11]	Mobile app for group event scheduling; study of 500+ users and 300 events	Invitations, time/venue suggestions, group voting, usage-log analysis	Covers only time and venue decisions; no budget, expenses, or tasks
2013	Kim et al. [12]	Community-informed conference scheduling (Cobi)	Community-sourcing, constraint solving, visual scheduling interface	Built for large academic conferences and professional organizers
2010	Škrabálek et al. [13]	Web service for collaborative organization of academic events (Takeplace)	Rich internet application, Web 2.0, online project management	Aimed at public professional events; no shared expense tracking
1996	Sandhu et al. [14]	Family of RBAC reference models	Users-roles-permissions separation, role hierarchies, constraints	Enterprise-wide roles; no per-object role scoping
2001	Ferraiolo et al. [15]	Proposed NIST RBAC standard	Core, hierarchical, and constrained RBAC	Specification only; leaves enforcement mechanism to implementers

2.3.2 Related Systems / Applications

Eventbrite [5], [9]. Launched in 2006, Eventbrite is a large platform for publishing public events, selling tickets, and tracking registrations. Its strengths are wide reach and polished ticketing and check-in tools. It has no shared expense tracking or task management, and its model is built around public attendees rather than a private family or friend group coordinating internally.

Splitwise [10]. Released in 2011, Splitwise records shared expenses within a group and calculates who owes whom, with support for multiple currencies. It settles debts well, but

it has no concept of an event with a date, guests, or tasks, and many convenience features on its free tier are limited.

Aisle Planner [7]. Aisle Planner, available since 2013, is a professional wedding-planning suite with vendor coordination, budgeting, guest management, and timelines. It is comprehensive, but it is priced and designed for professional planners, has a steep learning curve for ordinary families, and is limited to weddings.

WhatsApp and Google Sheets [6]. In practice, most groups combine a WhatsApp group with a shared spreadsheet. These tools are free, familiar, and need no setup. However, they provide no structure: decisions are buried in long message threads, anyone with the sheet link can change anything, and there is no automatic budget total, RSVP tracking, or task status. Table 2.2 summarizes these systems.

Table 2.2: Summary of Existing Systems

System	Year	Features	Strength	Limitations
Eventbrite [5], [9]	2006	Public event ticketing, registration, attendee tracking	Large reach, polished ticketing infrastructure	No expense tracking or task management; built for public, not private, events
Splitwise [10]	2011	Shared expense tracking, debt splitting, group finance	Clean debt settlement, multi-currency support	No events, guest lists, or tasks; free tier limited
Aisle Planner [7]	2013	Wedding planning, vendor coordination, budget, guest management	Comprehensive, professional-grade tools	Priced for professionals; steep learning curve; weddings only
WhatsApp + Google Sheets [6]	N/A	Messaging, manual data entry, file sharing	Free, familiar, no setup	No structure or role control; decisions get lost; data easily overwritten

2.4 Comparison of Existing Systems

Table 2.3 compares the existing systems with Festara across the features that matter most to a small group planning a private event. No single existing system offers event creation, per-event roles, guest RSVP, shared expense tracking against a budget, and task assignment together at no cost to the group.

Table 2.3: Feature Comparison of Existing Systems and Festara

Feature	Eventbrite	Splitwise	Aisle Planner	WhatsApp + Sheets	Festara
Private event creation	Partial	No	Yes	Manual	Yes
Per-event roles (Admin / Member / Guest)	Organizer only	No	Planner / client	No	Yes
Guest list and RSVP via link	Yes (tickets)	No	Yes	Manual	Yes
Shared expense logging	No	Yes	Partial	Manual	Yes
Budget vs. spending with warning	No	No	Yes	Manual	Yes (80% alert)
Task assignment with deadlines	No	No	Yes	No	Yes
AI checklist / budget suggestion	No	No	No	No	Phase 2
Free for personal use	Free events only	Limited	No	Yes	Yes

2.5 Summary

The research reviewed shows that structured input from all participants improves group planning [11], [12], and that a single web workspace can serve an organizing team [13]. RBAC theory gives a sound basis for controlling who can do what [14], [15]. Existing applications, however, each handle only one part of the problem or are aimed at professionals. Festara's boundary is therefore set as a free, browser-based workspace for private events that combines guests, expenses, tasks, and per-event roles, enforced at the database level, with AI guidance added in the second phase.

Chapter 3

REQUIREMENTS SPECIFICATION

3.1 Introduction

This chapter specifies what Festara must do and the qualities it must have. It begins with the hardware and software interfaces the system depends on, then lists the functional requirements of every module, and presents the use case model with detailed descriptions of the most important use cases. It then defines measurable non-functional requirements, the resources required to build the system, the database requirements, and the feasibility of the project. Requirements for all eight modules are specified here, even though only the first three are implemented at this milestone, so that the design in Chapter 4 covers the complete system.

3.2 Interface Requirements

Festara is a pure web application. It has no custom hardware, so its interface requirements describe the devices that run the browser and the development tools, and the external software services that the application communicates with.

3.2.1 Hardware Interface Requirements

End users need only a device with a modern web browser and an internet connection. Development requires an ordinary laptop. Server hardware is fully managed by Vercel and Supabase. Table 3.1 lists the hardware requirements.

Table 3.1: Hardware Interface Requirements

Component	Minimum Specification	Purpose
Development machine	Intel Core i5 / Ryzen 5 or equivalent, 8 GB RAM, 256 GB SSD	Running the Next.js development server, code editor, and browser testing
End-user device	Any desktop, laptop, tablet, or smartphone with a current browser; screen width 360 px or more	Accessing Festara
Network	Stable internet connection (4G or broadband)	All data is stored in the cloud; no offline mode in the first release
Server infrastructure	Managed by Vercel and Supabase	Hosting, database, authentication; no dedicated hardware

3.2.2 Software Interface Requirements

The software stack and the external services the application connects to are listed in Table 3.2. Communication with Supabase uses the official supabase-js client over HTTPS, and

real-time updates use a WebSocket connection [1].

Table 3.2: Software Interface Requirements

Software / Service	Version / Type	Role in Festara
Next.js [2]	16 (App Router)	Web framework: routing, server components, server actions
React	19	User interface library used by Next.js
TypeScript	5.x	Typed language for application code
Tailwind CSS	4.x	Utility-first styling and responsive layout
Supabase [1]	Managed cloud	PostgreSQL database, Auth, Realtime, Row Level Security
Vercel [2]	Hobby / cloud	Build, hosting, and continuous deployment from GitHub
Node.js	18 or above	JavaScript runtime for development and builds
OpenAI API [3]	Chat Completions / Structured Outputs	AI checklist and budget estimator (Phase 2)
Vercel AI SDK [4]	Latest	Streaming AI responses to the interface (Phase 2)
Git and GitHub	Latest	Version control and collaboration
Web browser	Chrome, Edge, Firefox, Safari (current versions)	Client runtime

3.3 Functional Requirements

Functional requirements describe what the system must do. They were derived from the modules defined in the project proposal and refined in meetings with the supervisor. Each requirement has an identifier that is reused in the use cases and test cases. Table 3.3 lists all functional requirements with their status at the 40 percent milestone.

Table 3.3: Functional Requirements

ID	Module	Requirement	Status
FR-01	Authentication	The system shall allow a user to register with full name, email, and password.	Completed
FR-02	Authentication	The system shall authenticate users with email and password and maintain a secure session.	Completed
FR-03	Authentication	The system shall allow a user to reset a forgotten password through an emailed link.	Completed
FR-04	Authentication	The system shall allow a user to view and update their name and contact number.	Completed

ID	Module	Requirement	Status
FR-05	Event Management	The system shall allow a logged-in user to create an event with name, type, date, location, description, and total budget.	Completed
FR-06	Event Management	The system shall add the creator of an event as its Admin automatically.	Completed
FR-07	Event Management	The system shall allow only the Admin to edit event details.	Completed
FR-08	Event Management	The system shall allow only the Admin to delete an event, after a confirmation prompt.	Completed
FR-09	Event Management	The system shall show a home dashboard listing every event the user belongs to with their role.	Completed
FR-10	Access Control	The system shall allow the Admin to generate an invite link that assigns a chosen role (Member or Guest).	Completed
FR-11	Access Control	The system shall add a logged-in user to the event with the link's role when they open a valid invite link.	Completed
FR-12	Access Control	The system shall reject expired or unknown invite links with a clear message.	Completed
FR-13	Access Control	The system shall allow the Admin to change a member's role or remove them from the event.	Completed
FR-14	Access Control	The system shall restrict every read and write operation according to the user's role in that event.	Completed
FR-15	Guests & RSVP	The system shall allow Admins and Members to add guests with name, contact number, and family side.	Planned
FR-16	Guests & RSVP	The system shall let guests confirm or decline attendance through an invite link.	Planned
FR-17	Guests & RSVP	The system shall show totals of invited, confirmed, declined, and pending guests, and filter by side or status.	Planned
FR-18	Expenses & Budget	The system shall allow Admins and Members to log an expense with amount, category, description, and date.	Planned
FR-19	Expenses & Budget	The system shall display total budget, total spent, and remaining balance, updated in real time.	Planned
FR-20	Expenses & Budget	The system shall display a warning when total spending crosses 80 percent of the budget.	Planned
FR-21	Expenses & Budget	The system shall show an expense breakdown by member and by category.	Planned
FR-22	Tasks	The system shall allow the Admin to create tasks with title, description, assignee, and deadline.	Planned
FR-23	Tasks	The system shall allow the assignee to move a task between To Do, In Progress, and Done.	Planned

ID	Module	Requirement	Status
FR-24	Tasks	The system shall show each member a personal view of their assigned tasks.	Planned
FR-25	Admin Dashboard	The system shall show event name, date, days remaining, member count, guest count, task completion, and budget usage on one screen.	Planned
FR-26	Admin Dashboard	The system shall record and display a recent activity log of who changed what and when.	Planned
FR-27	AI Features	The system shall generate a preparation checklist with suggested timelines from an event type and guest count.	Phase 2
FR-28	AI Features	The system shall suggest a budget split by category for an event type and expected attendance.	Phase 2
FR-29	AI Features	The system shall flag a category whose spending is significantly above typical levels for the event type.	Phase 2

3.4 Use Case Model

Festara has three actors. The **Admin** is the user who created an event, or who has been promoted to Admin; they control the event, its members, its budget, and its settings. The **Member** contributes to the event by adding guests, logging expenses, and updating their tasks. The **Guest** is invited only to confirm attendance and view basic event details. Every Admin can also perform all Member use cases. Figure 3.1 shows the use case diagram for the complete system; use cases marked with an asterisk belong to the second phase.

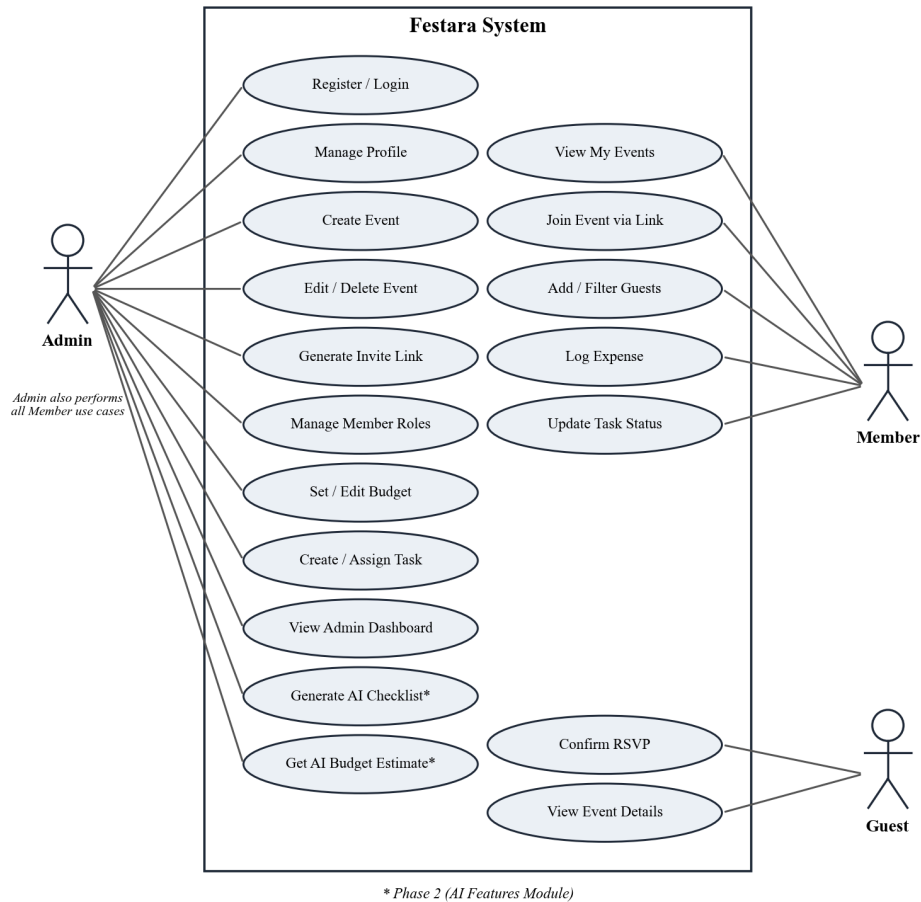


Figure 3.1: Use case diagram of Festara

3.5 Use Cases

Simple use cases such as login and profile update do not need full descriptions. The five use cases below were selected for full-dress description because they carry the core logic of the system: creating the event that everything else depends on, granting access through invite links, controlling roles, logging money against the budget, and recording guest responses.

3.5.1 Use Case 1: Create Event

Creating an event is the starting point for every other feature, and it is where the first role assignment happens. Table 3.4 describes it.

Table 3.4: Full Dress Use Case for Create Event

Use Case ID / Name	UC-01: Create Event
Scope	Festara web application
Level	User goal

Primary Actor	Registered user (becomes Admin)
Stakeholders and Interests	Organizer: wants a shared space ready quickly. Future members: want accurate event details.
Preconditions	User is logged in.
Success Guarantee	A new event is stored and the user is recorded as its Admin in event_members.
Main Success Scenario	1. User selects "Create Event" on the home dashboard. 2. System shows the event form. 3. User enters name, type, date, location, description, and total budget. 4. System validates the input. 5. System saves the event and adds the user as Admin. 6. System opens the new event's page.
Extensions	4a. A required field is empty or the budget is negative: system highlights the field and keeps the entered data. 4b. The date is in the past: system shows a warning and asks the user to confirm.
Special Requirements	Form must work on a 360 px wide screen; save must complete within 2 seconds on a 4G connection.
Related Requirements	FR-05, FR-06, FR-09

3.5.2 Use Case 2: Join Event via Invite Link

Invite links are the only way users gain access to an event, so this use case is central to access control. It is detailed in Table 3.5.

Table 3.5: Full Dress Use Case for Join Event via Invite Link

Use Case ID / Name	UC-02: Join Event via Invite Link
Scope	Festara web application
Level	User goal
Primary Actor	Invitee (future Member or Guest)
Stakeholders and Interests	Admin: wants only invited people to join, with the right role. Invitee: wants to join with as little effort as possible.
Preconditions	Admin has generated an invite link for a role; invitee has the link.
Success Guarantee	Invitee is a member of the event with the role encoded in the link.
Main Success Scenario	1. Invitee opens the invite link. 2. System asks the invitee to log in or register if they are not signed in. 3. System checks that the token exists and has not expired. 4. System adds the invitee to event_members with the link's role. 5. System redirects the invitee to the event page showing the features allowed for that role.

Extensions	3a. Token unknown or expired: system shows "This invite link is invalid or has expired" and asks the user to contact the Admin. 4a. Invitee is already a member: system keeps the existing role and opens the event.
Special Requirements	Tokens must be random and unguessable; links expire after 7 days by default.
Related Requirements	FR-10, FR-11, FR-12

3.5.3 Use Case 3: Manage Member Roles

Admins must be able to correct mistakes and respond to changes, such as promoting a trusted relative or removing someone who left the trip. Table 3.6 describes this use case.

Table 3.6: Full Dress Use Case for Manage Member Roles

Use Case ID / Name	UC-03: Manage Member Roles
Scope	Festara web application
Level	User goal
Primary Actor	Admin
Stakeholders and Interests	Admin: wants control over who can change event data. Members: want their access to be fair and predictable.
Preconditions	Admin is logged in and is Admin of the event.
Success Guarantee	The member's role is updated or the member is removed, and the change takes effect on their next request.
Main Success Scenario	1. Admin opens the Members page of the event. 2. System lists members with their roles. 3. Admin selects a new role for a member, or selects Remove. 4. System asks for confirmation when removing. 5. System saves the change and refreshes the list.
Extensions	3a. Admin tries to demote or remove the last remaining Admin: system blocks the action and explains that every event needs at least one Admin. 5a. A non-Admin calls the action directly: the database policy rejects it.
Special Requirements	Role changes must be enforced by the database, not only hidden in the interface.
Related Requirements	FR-13, FR-14

3.5.4 Use Case 4: Log Expense

Expense logging is the most frequent action during an event and drives the budget warning. Its detailed description is given in Table 3.7.

Table 3.7: Full Dress Use Case for Log Expense

Use Case ID / Name	UC-04: Log Expense
Scope	Festara web application
Level	User goal
Primary Actor	Member or Admin
Stakeholders and Interests	Member: wants a quick record of what they paid. Admin: wants accurate totals and early warning of overspending.
Preconditions	User is a Member or Admin of the event; the event has a budget.
Success Guarantee	Expense is stored with the user as payer; totals and remaining balance are updated for all viewers.
Main Success Scenario	1. User opens the Expenses page. 2. User selects "Add Expense". 3. User enters amount, category, description, and date. 4. System validates and saves the expense. 5. System recalculates total spent and remaining balance. 6. System pushes the update to other open sessions in real time.
Extensions	4a. Amount is zero or negative: system rejects the entry. 5a. Total spent crosses 80 percent of the budget: system shows a warning banner to all members. 5b. Total spent exceeds the budget: system shows the overspent amount in red.
Special Requirements	Totals must be computed from stored data, never from client-side values.
Related Requirements	FR-18, FR-19, FR-20, FR-21

3.5.5 Use Case 5: Confirm RSVP

Guests interact with Festara only through this use case, so it must be simple. It is described in Table 3.8.

Table 3.8: Full Dress Use Case for Confirm RSVP

Use Case ID / Name	UC-05: Confirm RSVP
Scope	Festara web application
Level	User goal
Primary Actor	Guest
Stakeholders and Interests	Guest: wants to reply quickly. Admin: wants an accurate headcount for catering and seating.
Preconditions	Guest has a valid guest invite link.
Success Guarantee	Guest's RSVP status is stored as Confirmed or Declined and reflected in the guest summary.

Main Success Scenario	1. Guest opens the invite link. 2. System shows event name, date, and location. 3. Guest selects Confirm or Decline. 4. System saves the response and shows a thank-you message.
Extensions	3a. Guest changes their mind later: guest reopens the link and changes the response before the event date.
Special Requirements	Page must load and be usable on a low-end smartphone.
Related Requirements	FR-16, FR-17

3.6 Non-Functional Requirements

Non-functional requirements describe how well the system performs its functions. Only the qualities relevant to Festara are listed, and each one has a measurable target so that it can be verified in Chapter 6.

3.6.1 Performance

Groups often use Festara on mobile data during the event itself, so pages must load quickly. The event dashboard shall reach Largest Contentful Paint in under 2.5 seconds on a simulated 4G connection, and create, update, and delete actions shall complete in under 1 second at the 95th percentile, measured with Lighthouse and Vercel analytics.

3.6.2 Reliability

Lost expense records would undermine trust in the whole system. Festara inherits the availability of Vercel and Supabase and targets at least 99.5 percent monthly availability. Every write shall either complete fully or fail with a visible error message, with no partially saved records; multi-step operations such as accepting an invite run inside a single database function.

3.6.3 Security

Festara stores personal data such as phone numbers and spending. All traffic shall use HTTPS. Passwords are hashed and managed by Supabase Auth and are never stored by the application [1]. Row Level Security shall be enabled on 100 percent of application tables, and a user who is not a member of an event shall receive zero rows when querying it directly. Invite tokens shall contain at least 128 bits of randomness and expire after 7 days by default.

3.6.4 Usability

Many users will be first-time, non-technical organizers. A new user shall be able to register, create an event, and share an invite link within 3 minutes without help, and the system shall achieve a System Usability Scale score of at least 70 in a trial with at least 10 users.

3.6.5 Consistency

All members must see the same numbers. Budget totals, guest counts, and task completion percentages shall be calculated from the database on each request or through real-time subscriptions, never cached on one client, so that two members viewing the same event at the same time see identical values within 2 seconds of a change.

3.6.6 Portability

The application shall work on the current versions of Chrome, Edge, Firefox, and Safari, and its layout shall remain usable on screen widths from 360 px to 1920 px, with no horizontal scrolling.

3.7 Resource Requirements

3.7.1 Equipment

The hardware and software tools used by the team, with the reason each was chosen, are given in Table 3.9.

Table 3.9: Equipment and Tools with Justification

Resource	Justification
Laptops (8 GB RAM or more), one per member	Sufficient for the Next.js dev server, editor, and browser; already owned by the team
Next.js 16 with TypeScript	Server components reduce client-side JavaScript; server actions remove the need for a separate API layer; types catch errors early
Supabase	Provides PostgreSQL, Auth, Realtime, and RLS in one free tier, removing the need to build and host a custom backend
Vercel	Native hosting for Next.js with automatic preview deployments for every pull request
Tailwind CSS	Fast, consistent, responsive styling without writing large stylesheets
GitHub	Shared repository, pull requests, and issue tracking for the backlog
Figma	High-fidelity prototypes reviewed with the supervisor before coding
OpenAI API (Phase 2)	Structured outputs return checklists and budget splits as reliable JSON [3]

3.7.2 Funds

The project is designed to run within free tiers. Supabase's free plan and Vercel's Hobby plan cover development and the expected demo usage. The only expected cost is OpenAI API usage in the second phase, which is billed per request and is expected to stay within a few US dollars during development and testing. A custom domain is optional.

3.7.3 Human Effort

The work is divided among the three team members as shown in Table 3.10. The total effort is estimated at 7.5 person-months over the two semesters of the project.

Table 3.10: Task Division and Human Effort

Team Member	Responsibilities	Effort (person-months)
Anas Altaf (MC-331), Team Lead	Architecture, database schema and RLS policies, RBAC and invite links, deployment, Expense and Budget module, sprint planning	2.5
Hammad Ansar (MC-304)	Authentication module, UI design in Figma, Tailwind components, Guest List and RSVP module, Admin Dashboard	2.5
Muhammad Sami Ullah (MC-336)	Event Management module, Task module, AI Features module, testing, and documentation	2.5
Total		7.5

3.8 Database Requirements

Festara needs a relational database because its data is highly connected: every guest, expense, task, and invitation belongs to an event, and every membership links a user to an event with a role. PostgreSQL, provided by Supabase, was selected because it supports foreign keys, enumerated types, transactions, and Row Level Security. The database shall:

- store user profiles linked one-to-one with Supabase Auth users;
- store events and the role of each user in each event, with a unique constraint so that a user holds only one role per event;
- store invitations with a unique random token, the role granted, and an expiry date;
- store guests, expenses, and tasks, each linked to an event by a foreign key with cascading delete, so that deleting an event removes all its data;
- use enumerated types for event type, role, RSVP status, expense category, and task status, so that invalid values cannot be stored;
- enforce access through RLS policies on every table, using a helper function that checks the current user's role in the event.

The complete schema is shown in the entity relationship diagram in Section 4.7, and each table is described in the data dictionary in Appendix I.

3.9 Project Feasibility

3.9.1 Technical Feasibility

The project is technically feasible. Every component uses mature, well-documented technology [1]–[4]. Supabase removes the need to build authentication and database hosting from scratch, and Next.js server actions remove the need for a separate API server. The team has prior coursework experience with React, SQL, and JavaScript. The system runs in any modern browser and does not need a powerful machine on either the development or the user side.

3.9.2 Operational Feasibility

Operationally, Festara replaces habits users already have rather than asking them to learn a new process. An organizer creates an event, shares a link in the family WhatsApp group as they already do, and members log expenses the moment they pay. The three roles mirror how groups already organize themselves. Because the application is web-based, there is nothing to install, which removes the largest barrier for older family members.

3.9.3 Economic Feasibility

Development uses free tiers of Supabase and Vercel and free open-source frameworks. The only variable cost, AI requests in the second phase, is small and can be limited per event. Festara is therefore economically feasible for a student project and remains free to its end users.

3.9.4 Legal and Ethical Feasibility

The system is legally and ethically feasible. It does not process payments, so it is not subject to financial licensing. It collects only the personal data needed to coordinate an event (name, email, phone number, and spending entries) and makes that data visible only to members of the same event through database-enforced policies. Users can delete their events, which removes all related records. Users with strict privacy requirements are informed that data is stored on cloud servers. The system is designed to help people and contains no feature that could be used to harm them.

3.10 Summary

This chapter defined 29 functional requirements across eight modules, of which the 14 requirements for authentication, event management, and access control are complete. Five use cases were described in full, and six measurable non-functional requirements were set

for performance, reliability, security, usability, consistency, and portability. The resource, database, and feasibility analysis shows that the project can be completed with free tools and the team's existing skills.

Chapter 4
SYSTEM MODELLING

4.1 Introduction

System modelling turns the requirements of Chapter 3 into a design that can be built and checked. Diagrams make it possible to find gaps, such as a missing permission check or an orphaned table, before any code is written, and they give all three team members the same picture of the system. This chapter presents the overall architecture, the design approach, the interface design, data flow diagrams at levels 0 and 1, the 4+1 architectural views (logical, process, development, and physical), and the entity relationship diagram. The models cover the complete system, including modules that will be implemented after the 40 percent milestone.

4.2 System Design

Festara uses a three-tier, serverless architecture, shown in Figure 4.1. The presentation tier is the user's browser, which renders pages produced by Next.js. The application tier is the Next.js application hosted on Vercel; React Server Components fetch data on the server, and server actions handle every create, update, and delete request after validating input. The data tier is Supabase, which provides authentication, the PostgreSQL database protected by Row Level Security, and a Realtime service that pushes changes to open browser sessions. In the second phase, an AI route in the application tier will call the OpenAI API and stream responses back through the Vercel AI SDK [3], [4].

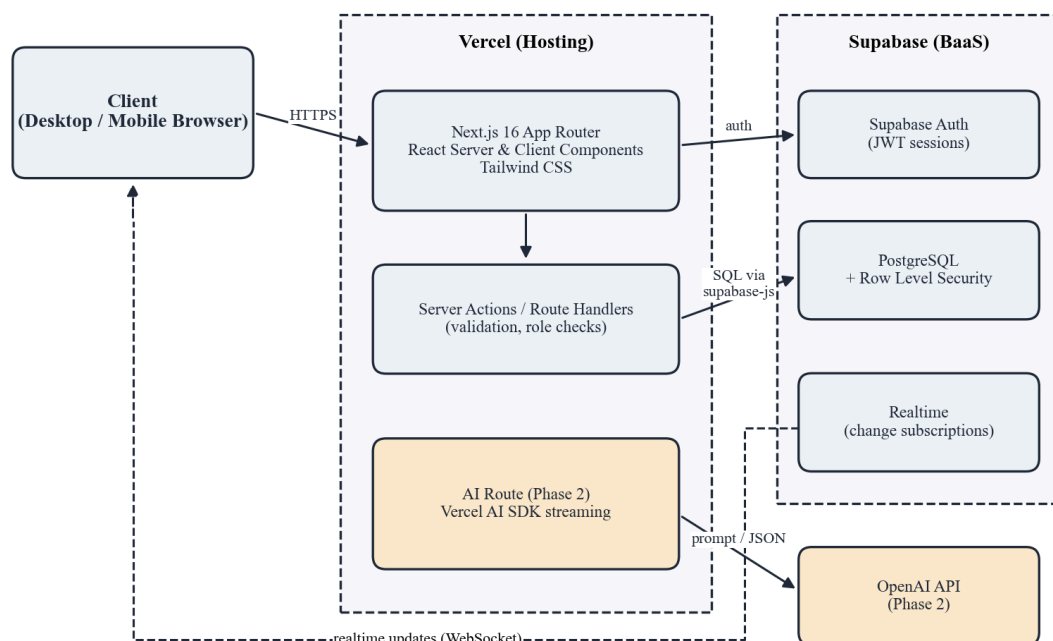


Figure 4.1: System architecture of Festara

A key design decision is that authorization lives in the data tier. The application tier also checks roles to show helpful messages, but the final decision is made by the database policies. This means that even if a user calls the Supabase API directly with their own session token, they cannot read or change data in an event they do not belong to [17].

4.3 Design Approach

The team adopted a **top-down design approach**. The system was first defined as a whole in the proposal, then decomposed into eight modules, then each module was broken into pages, server actions, and database tables. This approach suits Festara because the modules share a common core, the event and its members, that had to be designed first so that every later module could attach to it. Shared building blocks such as the Supabase client, the role-checking helper, and form validation schemas were identified during this decomposition and built once in a common library folder.

4.4 Interface Design

The interface was designed mobile-first, because most members will log expenses and check tasks on their phones. Navigation within an event uses a tab bar for Overview, Guests, Expenses, Tasks, and Members, and the tabs a user sees depend on their role. The visual style uses a light background, a single accent color, and large touch targets of at least 44 px.

4.4.1 High Fidelity Prototype

High-fidelity prototypes of the key screens were created in Figma and reviewed with the supervisor before development. Figure 4.2 shows the login and registration screen, Figure 4.3 shows the home dashboard listing the user's events, and Figure 4.4 shows the members and invite screen used by the Admin.

[Insert screenshot: Figma prototype – Login / Register screen]

Figure 4.2: Prototype of the login and registration screen

[Insert screenshot: Figma prototype – Home dashboard (My Events)]

Figure 4.3: Prototype of the home dashboard

[Insert screenshot: Figma prototype – Event members and invite link]

Figure 4.4: Prototype of the members and invite screen

4.5 Data Flow Diagrams

Data flow diagrams show how information moves between external entities, processes, and data stores, without showing control logic. Figure 4.5 is the level 0 (context) diagram.

It treats Festara as a single process exchanging data with four external entities: the Admin, who supplies event details, budgets, role changes, and invites; the Member, who supplies guests, expenses, and task updates; the Guest, who returns RSVP responses; and the OpenAI API, which returns checklists and estimates in the second phase.

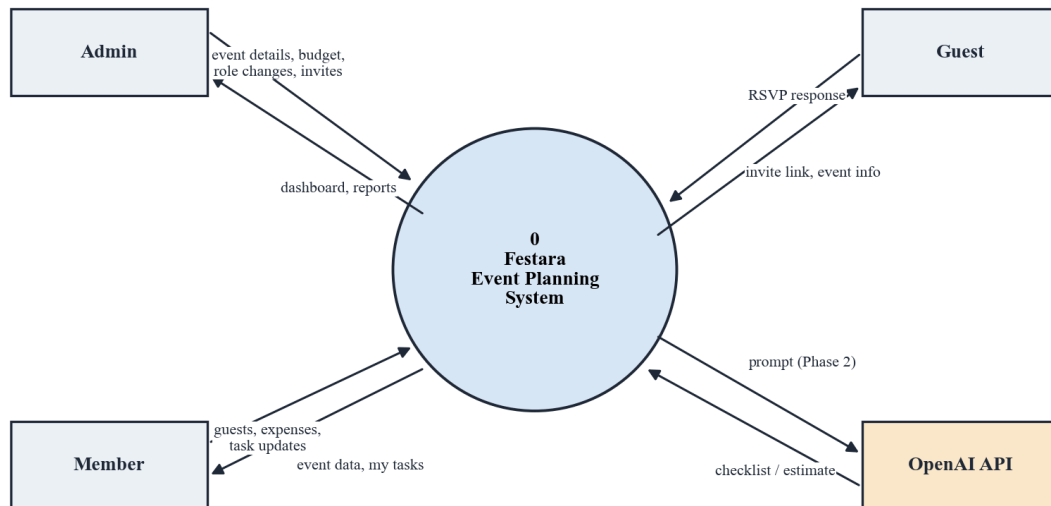


Figure 4.5: DFD level 0 (context diagram) of Festara

Figure 4.6 decomposes the system into six level 1 processes: authenticate user, manage event, manage roles and invites, manage guests and RSVP, track expenses and budget, and manage tasks. Each process reads from and writes to its own data store, and processes 4.0, 5.0, and 6.0 first perform a role check against the event_members store before accepting data.

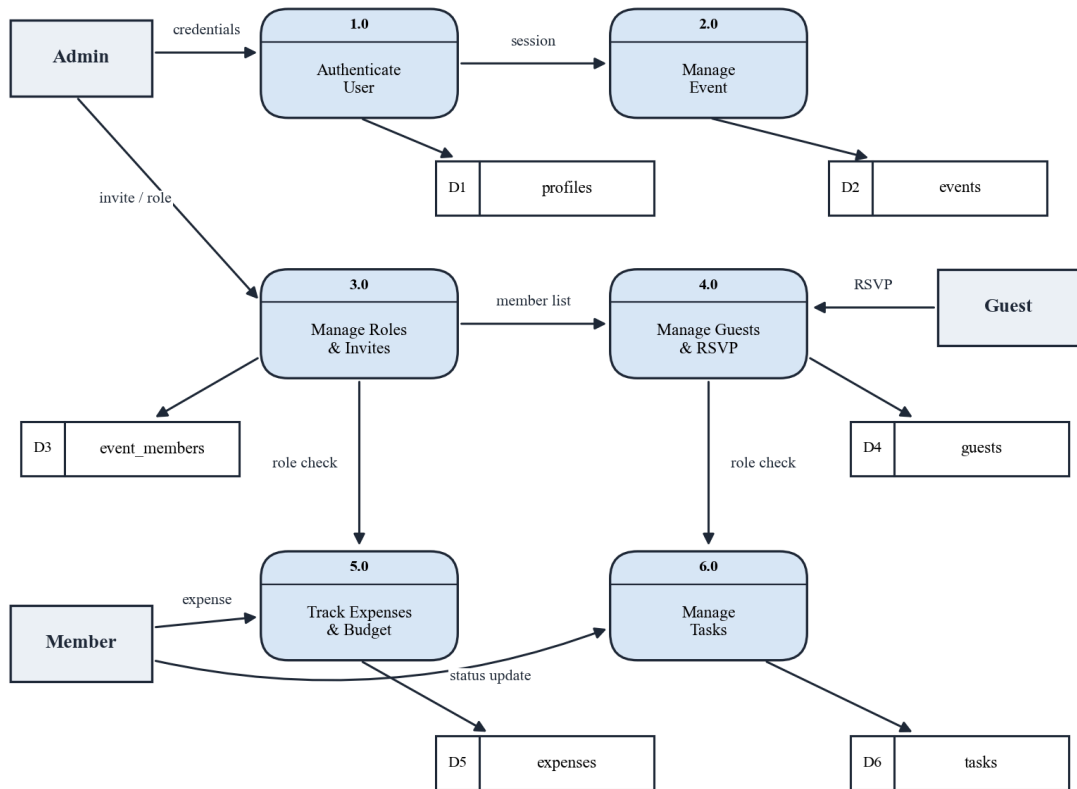


Figure 4.6: DFD level 1 of Festara

4.6 4+1 View Model of Architecture

The 4+1 view model describes a system's architecture from several viewpoints, each aimed at a different concern, tied together by the use case scenarios of Section 3.5. The logical, process, development, and physical views of Festara are presented below; the use case view was given in Figure 3.1.

4.6.1 Logical View

The logical view shows the main classes of the domain and their relationships. As shown in Figure 4.7, a User creates many Events, and the link between them is the EventMember class, which holds the user's role and provides the can(action) check used throughout the system. An Event issues Invitations and owns its Guests, Expenses, and Tasks. BudgetSummary is a computed class that aggregates expenses to give the remaining balance and the usage percentage that triggers the 80 percent warning. ActivityLog records every change for the Admin dashboard.

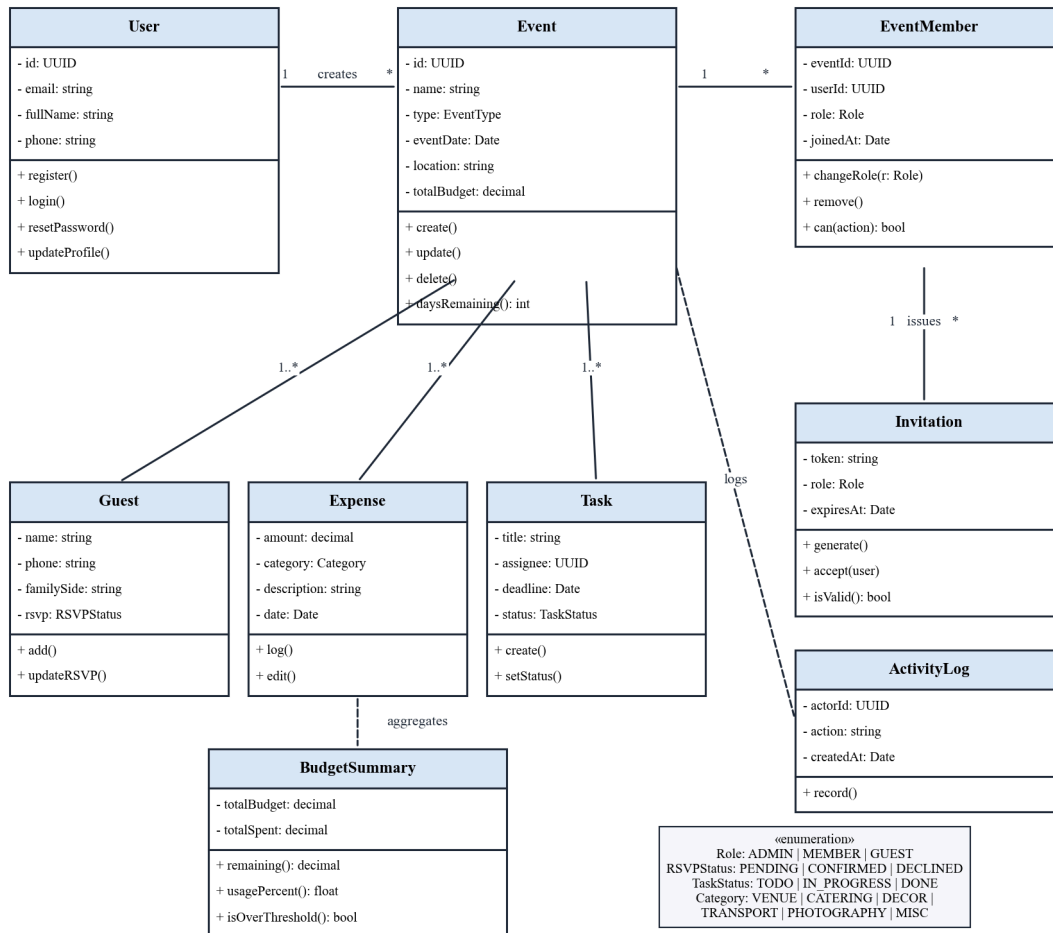


Figure 4.7: Class diagram of Festara

4.6.2 Process View

The process view describes the system's behavior at run time. Figure 4.8 is an activity diagram of the most important workflow at this milestone: an Admin creating an event and an invitee joining it through a link. It shows the two decision points, form validation and token validation, and the alternate paths when either fails.

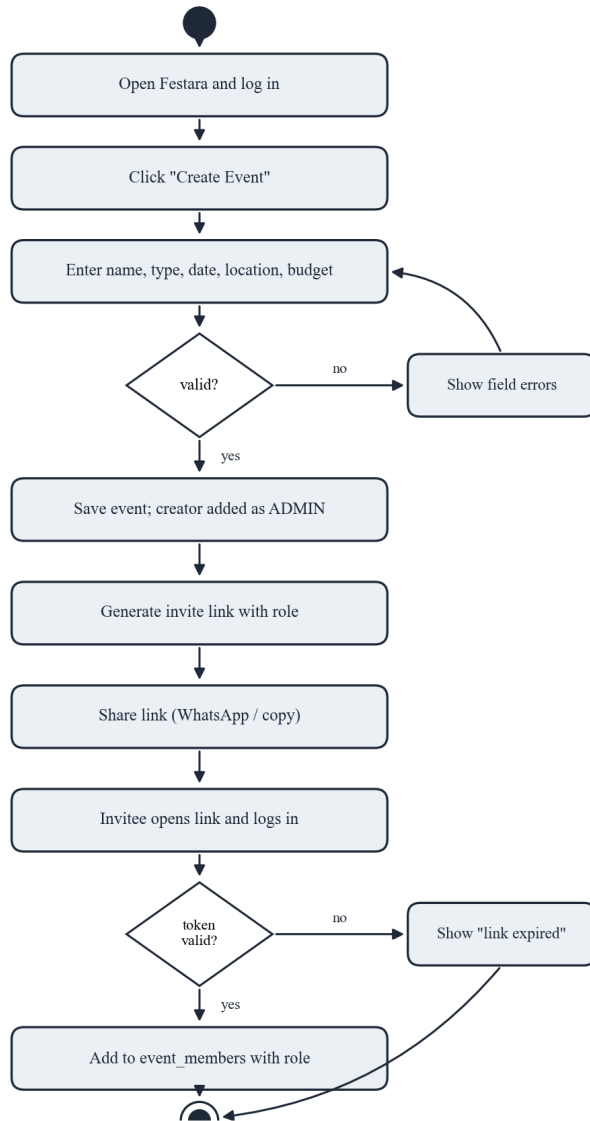


Figure 4.8: Activity diagram for creating an event and joining through an invite link

Figure 4.9 is a state diagram for a task. A task is created in the To Do state, moves to In Progress when the assigned member starts it, can move back if paused, and reaches Done when completed. An Admin can reopen a completed task.

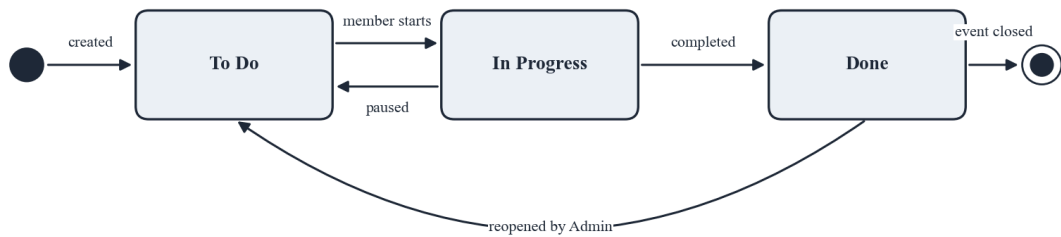


Figure 4.9: State diagram of a task

Figure 4.10 is a sequence diagram for accepting an invitation. The browser sends the token to a server action, which obtains the current user from Supabase Auth and then calls a database function. The function looks up the invitation, checks its expiry, and inserts the membership in one transaction. Row Level Security confirms the insert before the user is redirected to the event.

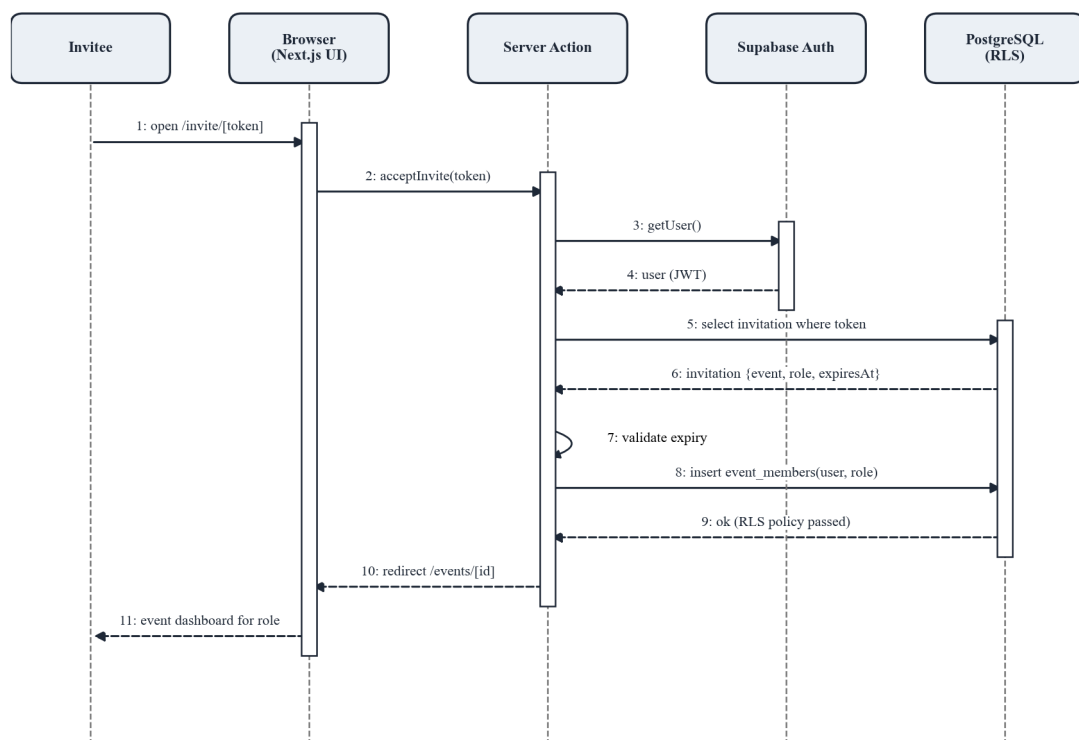


Figure 4.10: Sequence diagram for accepting an invitation

4.6.3 Development View

The development view shows how the software is organized into components. As Figure 4.11 shows, the Next.js application is split into user interface components for each

module, which all depend on a shared library containing the Supabase client, the role guard, validation schemas, and server actions. The library is the only component that talks to Supabase Auth and the database. The AI module is a separate component that will call the OpenAI API.

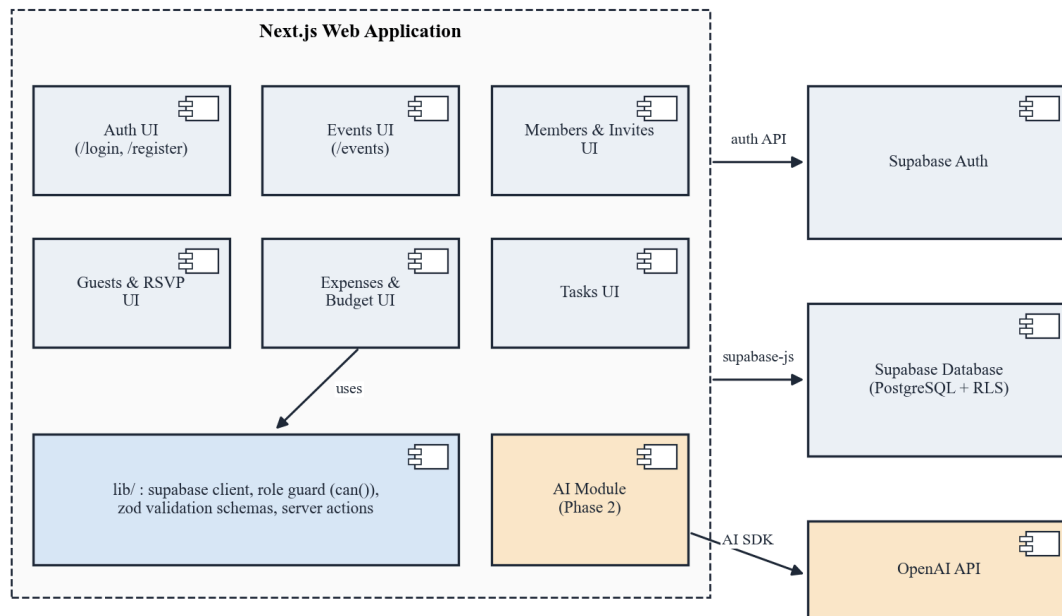


Figure 4.11: Component diagram of Festara

4.6.4 Physical View

The physical view shows where the software runs. As Figure 4.12 shows, the user's browser downloads the Festara interface from Vercel, where the Next.js build and server actions run on Vercel's edge and serverless infrastructure. Server actions communicate with the managed Supabase project over HTTPS, and browsers keep a secure WebSocket connection to Supabase Realtime for live updates.

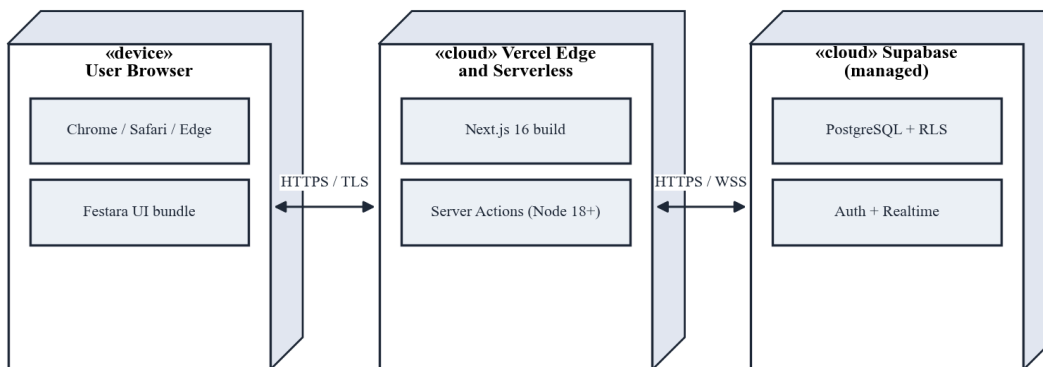
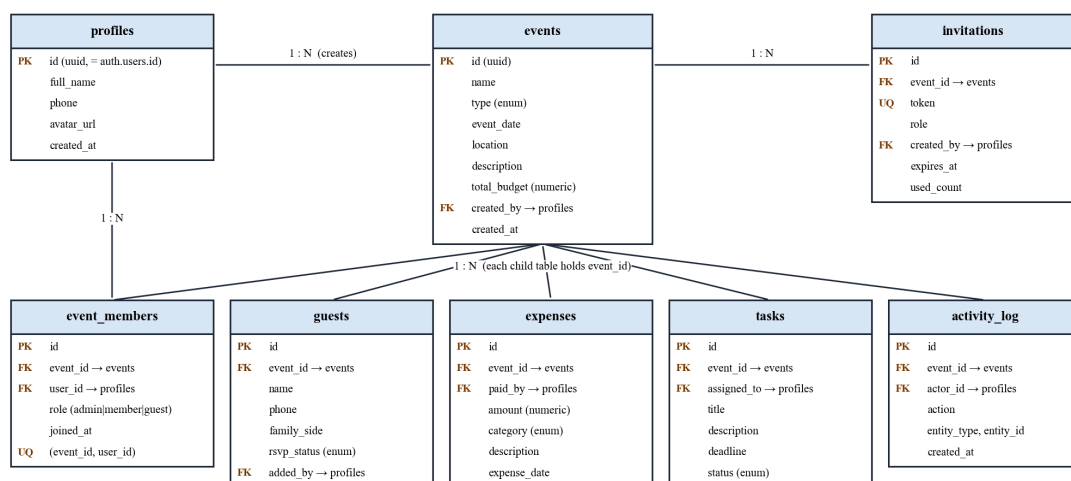


Figure 4.12: Deployment diagram of Festara

4.7 Entity Relationship Diagram

Figure 4.13 shows the database schema. The events table is the hub: event_members, invitations, guests, expenses, tasks, and activity_log each reference it through an event_id foreign key with cascading delete. The profiles table extends Supabase's built-in auth.users table and is referenced by the creator, member, payer, assignee, and actor columns. The pair (event_id, user_id) in event_members is unique, which guarantees a single role per user per event.



PK = primary key; FK = foreign key; UQ = unique. Row Level Security is enabled on every table.

Figure 4.13: Entity relationship diagram of Festara

4.8 Summary

This chapter presented Festara's three-tier serverless architecture and the top-down design approach used to decompose it. The interface design was shown through high-fidelity prototypes, data movement through level 0 and level 1 DFDs, and the architecture through the logical, process, development, and physical views. The entity relationship diagram defines a schema in which every record belongs to an event and every access is checked against the user's role in that event.

Chapter 5

IMPLEMENTATION

5.1 Introduction

This chapter describes how the design of Chapter 4 has been implemented up to the 40 percent milestone. It first describes the development environment and project structure, then explains the three completed modules, User Authentication, Event Management, and Role-Based Access Control, including the key algorithms and the libraries and services used. It then summarizes the implementation plan for the remaining modules.

5.2 Development Environment and Project Structure

The application is a single Next.js 16 project written in TypeScript and kept in a shared GitHub repository. Every push to the main branch deploys automatically to Vercel, and every pull request receives its own preview URL that the team uses for review. The database schema is managed as SQL migration files in the repository, so that the schema can be recreated in a fresh Supabase project. Table 5.1 shows the main folders.

Table 5.1: Project Folder Structure

Folder	Contents
app/(auth)/	Login, register, forgot-password, and reset-password pages
app/events/	Home dashboard, create-event form, and the [id] route for each event with its tabs
app/invite/[token]/	Invite acceptance page
components/	Reusable UI components: buttons, forms, cards, dialogs, role badges
lib/supabase/	Server and browser Supabase clients and the session middleware
lib/actions/	Server actions for profiles, events, invitations, and members
lib/validation/	Zod schemas shared by forms and server actions
supabase/migrations/	SQL files for tables, enums, functions, and RLS policies

5.3 Modules of the FYP

5.3.1 User Authentication Module

Authentication is implemented with Supabase Auth using email and password [1]. When a user registers, Supabase creates a record in `auth.users` and sends a confirmation email; a database trigger then creates a matching row in the `profiles` table with the user's full name. Login returns a session that is stored in secure, HTTP-only cookies by the `@supabase/ssr` helper, and a Next.js middleware refreshes the session on each request so that server

components always know the current user. Pages under /events redirect to /login when there is no session, and the original destination is kept so the user returns to it after logging in. Password reset uses Supabase's emailed recovery link, which opens the reset page where the user sets a new password. The profile page lets users update their name and phone number through a server action that validates the input with a Zod schema. Figure 5.1 shows the implemented login screen.

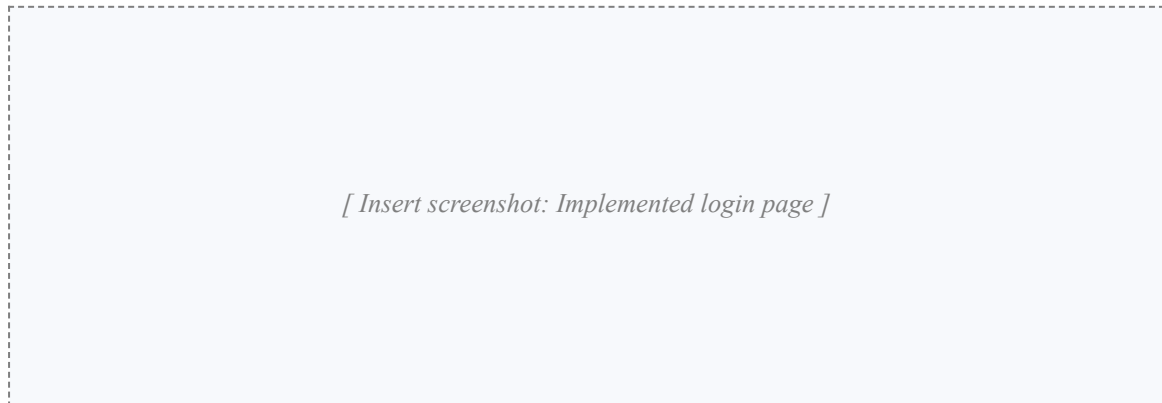


Figure 5.1: Implemented login screen

5.3.2 Event Management Module

The home dashboard is a server component that queries the events joined through `event_members` for the current user and displays each event as a card with its name, type, date, days remaining, and the user's role badge. Because of Row Level Security, the query only ever returns the user's own events, so no filtering by user is needed in the application code.

The create-event form collects the name, type, date, location, description, and total budget. Event type is selected from a fixed list (Wedding, Engagement, Mehndi, Walima, Birthday, Eid Gathering, Trip, University Event, and Other) that matches a PostgreSQL enumerated type. The server action validates the input, inserts the event, and inserts the creator into `event_members` as Admin. Both inserts run in a single database function so that an event can never exist without an Admin. Editing and deleting are available only to Admins; deletion asks for confirmation by having the user type the event name, and cascading foreign keys remove all related data. Figure 5.2 shows the create-event form.

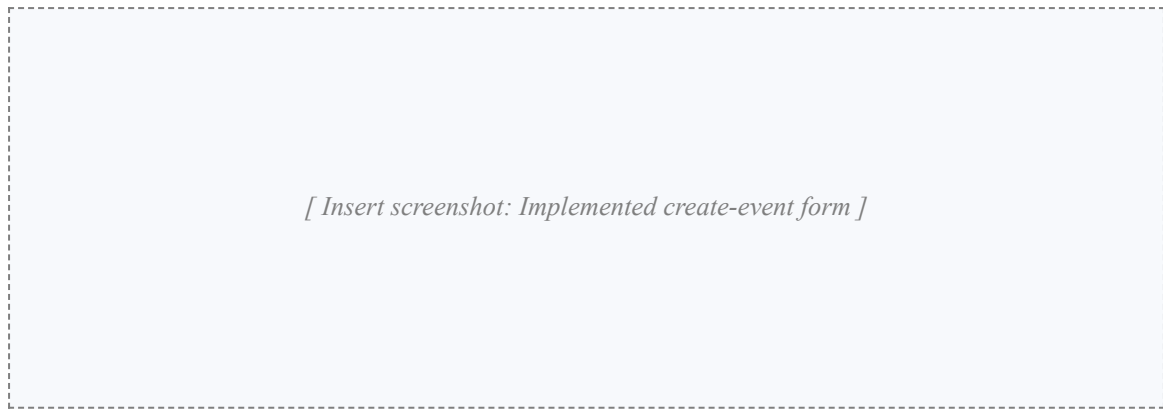


Figure 5.2: Implemented create-event form

5.3.3 Role-Based Access Control Module

Access control follows core RBAC [15], with roles scoped to an event. The permissions of each role are given in Table 5.2. Permissions are enforced in two layers: a `can(role, action)` helper in the application decides which buttons and tabs to show, and PostgreSQL Row Level Security policies make the final decision in the database [17].

Table 5.2: Role Permission Matrix

Action	Admin	Member	Guest
View event details	Yes	Yes	Basic only
Edit or delete event, set budget	Yes	No	No
Generate invite links	Yes	No	No
Change roles or remove members	Yes	No	No
Add guests	Yes	Yes	No
Log expenses	Yes	Yes	No
Create and assign tasks	Yes	No	No
Update status of own tasks	Yes	Yes	No
Confirm own RSVP	Yes	Yes	Yes
View admin dashboard and activity log	Yes	No	No

All policies use a single helper function, `has_role`, which checks whether the current user holds a given role in a given event. The function runs with security definer rights so that it can read `event_members` without triggering that table's own policies recursively. Figure 5.3 shows the function and two of the policies on the `events` table.

```

create function public.has_role(eid uuid, r public.member_role)
returns boolean language sql stable security definer
set search_path = public as $$
select exists (
select 1 from event_members
where event_id = eid and user_id = auth.uid()
and (role = r or role = 'admin'));
$$;
alter table events enable row level security;
create policy "members read their events" on events
for select using (exists (select 1 from event_members m
where m.event_id = events.id and m.user_id = auth.uid()));
create policy "only admins update events" on events
for update using (public.has_role(id, 'admin'));

```

Figure 5.3: Role helper function and Row Level Security policies on the events table

Invite link algorithm. When an Admin generates an invite, the server creates a token from 16 cryptographically random bytes (128 bits) encoded as a URL-safe string, stores it in the invitations table with the chosen role and an expiry 7 days ahead, and returns the link `/invite/[token]`. When the invitee opens the link, the steps are:

1. If the visitor is not logged in, redirect to `/login` and keep the invite URL as the return destination.
2. Call the database function `accept_invitation(token)`, which looks up the invitation and locks the row.
3. If no row is found, or `expires_at` is in the past, return an error; the page shows "This invite link is invalid or has expired."
4. If the user is already a member, keep their current role; otherwise insert a row into `event_members` with the invitation's role.
5. Increment `used_count`, commit the transaction, and redirect to the event page.

Running steps 2 to 5 in one database function keeps the operation atomic, so two people opening the link at the same moment cannot create inconsistent data. Figure 5.4 shows the server action that starts the process.

```

export async function acceptInvite(token: string) {
const supabase = await createClient();
const { data: { user } } = await supabase.auth.getUser();
if (!user) redirect(`/login?next=/invite/${token}`);
const { data: eventId, error } = await supabase
.rpc("accept_invitation", { p_token: token });
if (error) return { error: "This invite link is invalid or has expired." };
redirect(`/events/${eventId}`);
}

```

Figure 5.4: Server action for accepting an invitation

On the Members page, the Admin sees every member with a role selector and a remove button. The change-role and remove actions refuse to demote or remove the last Admin of

an event; this rule is checked both in the server action and in a database trigger. Figure 5.5 shows the implemented members screen.

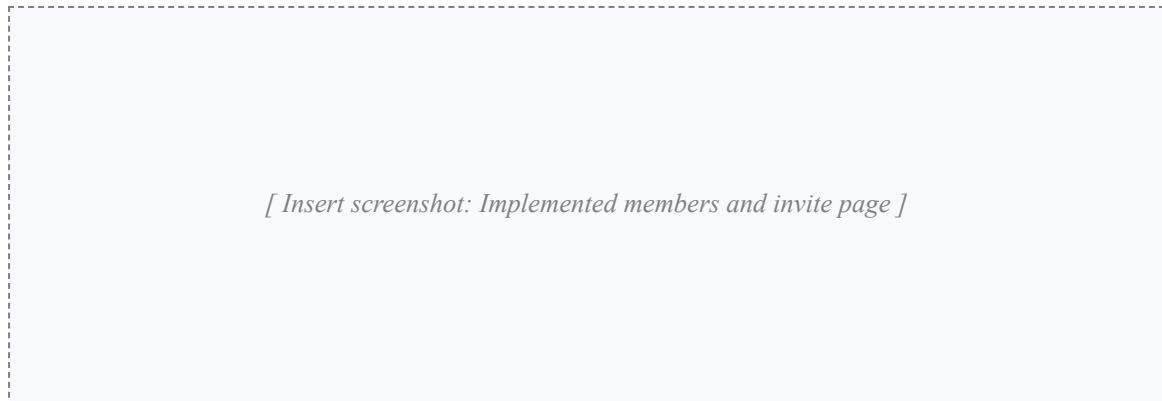


Figure 5.5: Implemented members and invite screen

5.3.4 Planned Implementation of Remaining Modules

The remaining modules reuse the same pattern of server actions, validation schemas, and RLS policies, so most of the infrastructure is already in place. Their planned implementation is summarized below.

- **Guest List and RSVP:** a guests table with RLS allowing Admins and Members to insert; a guest invite link that opens a public RSVP page; summary counts computed with a grouped SQL query; filters by family side and status.
- **Expense and Budget Tracking:** an expenses table with paid_by set from the session; a database view that returns total spent, remaining balance, and usage percentage; the warning appears when usage is at least 80 percent; Supabase Realtime subscriptions refresh totals on every open screen; breakdown charts by member and category.
- **Tasks and Arrangements:** a tasks table with a status enum; an update policy that lets the assignee change only the status column; a personal "My Tasks" view; completion percentage calculated as Done tasks divided by all tasks.
- **Admin Dashboard:** a single server component that runs the summary queries in parallel; an activity_log table filled by database triggers on insert, update, and delete.
- **AI Features (Phase 2):** a route handler that sends the event type and guest count to the OpenAI API with a JSON schema for structured output [3], streams the checklist to the browser with the Vercel AI SDK [4], and lets the Admin turn items into tasks with one click.

5.4 H/W Module Details

Festara is a software-only system and has no hardware module.

5.5 Summary

The foundation of Festara is in place. Users can register, log in, reset their password, and manage their profile; they can create, edit, and delete events; and access to every event is controlled by per-event roles, invite links, and database-enforced Row Level Security policies. The remaining modules will be built on this foundation using the same structure.

Chapter 6
RESULT / TESTING, ANALYSIS AND
VALIDATION

6.1 Introduction

This chapter describes how the implemented modules are tested and how the results are recorded. It explains the testing techniques chosen and the reason for each, presents the test cases for the authentication, event management, and access control modules, and sets out how the non-functional requirements of Section 3.6 will be measured. Test cases are traced to the functional requirements of Table 3.3.

6.2 Testing Strategy

6.2.1 Testing Environment

Tests are run on the deployed Vercel preview of each pull request, connected to a separate Supabase test project so that test data never mixes with demo data. Functional tests are run on a laptop with an Intel Core i5 processor and 8 GB RAM in the latest Chrome browser, and on an Android smartphone in Chrome to check mobile layouts.

6.2.2 Black Box Testing

Black box testing checks the system's behavior against its requirements without looking at the code. It is the main technique for Festara because the requirements in Table 3.3 describe visible behavior, such as a validation message or a redirect, that a tester can check directly. Each test case is executed by a team member who did not write the feature, using equivalence partitioning and boundary values for inputs such as the budget and the invite expiry date.

6.2.3 Unit Testing

Unit testing checks individual functions in isolation. The Zod validation schemas and the `can(role, action)` permission helper are tested with Vitest, because a mistake in these small functions would affect every module. Each role and action pair in Table 5.2 has a unit test.

6.2.4 Integration Testing

Integration testing checks that components work correctly together. The invite flow is the main target, because it crosses the browser, a server action, Supabase Auth, and a database function. The test creates two users, generates an invite as the first user, accepts it as the second, and checks the resulting membership and visible pages.

6.2.5 Security Testing of Access Policies

Because authorization is enforced by Row Level Security, the policies are tested directly against the database rather than only through the interface. A script signs in as users with each role, and as a user outside the event, and attempts every select, insert, update, and delete on each table using the Supabase client. The expected outcome for every attempt is derived from Table 5.2; any attempt that succeeds when it should fail is recorded as a defect.

6.3 Test Cases

Eleven test cases were designed for the modules completed at this milestone, prioritized by risk: tests for access control and invite handling were written first because a failure there would expose private data. Table 6.1 shows one test case in the full department format, and Table 6.2 summarizes all test cases. The Actual Result and Status columns are filled in as each test is executed.

6.3.1 Test Case TC-07: Join Event via Valid Invite Link

This test case verifies the most important flow at this milestone: that a second user can join an event with exactly the role chosen by the Admin. Its details are given in Table 6.1.

Table 6.1: Test Case TC-07 in Full Format

Test Stage	Integration
Test Date	__/__/__
Tester	Muhammad Sami Ullah
Test Case Number	TC-07
Test Case Description	Verifies that a user opening a valid Member invite link is added to the event as a Member.
Requirement(s) Tested	FR-10, FR-11, FR-14
Roles and Responsibilities	Tester executes steps; Anas Altaf (Team Lead) verifies database rows.
Set Up Procedures	Two registered test accounts (User A, User B) on the Supabase test project; User A has created an event.
Stop Procedures	Log out both users; delete the test event.
Hardware	Laptop (Core i5, 8 GB RAM)
Software	Chrome (latest), Vercel preview deployment, Supabase test project
Input Specifications	Event created by User A; invite role = Member; expiry = 7 days.

Procedural Steps	1. Log in as User A and open the event's Members page. 2. Generate a Member invite link and copy it. 3. Log out and log in as User B. 4. Open the copied link. 5. Check User B's dashboard and the event's Members page.
Expected Results	User B is redirected to the event; the event appears on User B's dashboard with a Member badge; User A sees User B listed as Member; User B cannot see the Edit Event button.
Actual Results	
Status (Pass / Fail)	

Table 6.2: Summary of Test Cases

ID	Req.	Test Case	Steps / Input	Expected Result	Actual Result	Status
TC-01	FR-01	Register with valid details	Enter new name, email, password (8+ chars); submit	Account created; confirmation email sent; profile row created		
TC-02	FR-02	Login with wrong password	Enter registered email and wrong password	Error "Invalid login credentials"; no session created		
TC-03	FR-03	Password reset	Request reset; open emailed link; set new password	Login succeeds with the new password only		
TC-04	FR-05, FR-06	Create event with valid data	Fill all fields; budget 50,000; submit	Event saved; creator listed as Admin; event page opens		
TC-05	FR-05	Create event with negative budget	Enter budget -100; submit	Validation error on budget field; nothing saved		
TC-06	FR-07, FR-14	Member tries to edit event	Log in as Member; call update action directly	Request rejected; event unchanged		
TC-07	FR-10, FR-11	Join event via valid invite link	Admin creates Member link; second user opens it	Second user added as Member; event visible on their dashboard		
TC-08	FR-12	Open expired invite link	Set expires_at in the past; open link	Message "invalid or has expired"; no membership created		
TC-09	FR-13	Admin demotes the last Admin	Sole Admin selects Member for themselves	Action blocked with explanation; role unchanged		

ID	Req.	Test Case	Steps / Input	Expected Result	Actual Result	Status
TC-10	FR-14	Non-member reads event directly	Using a valid session of a non-member, query events by id via Supabase API	Zero rows returned		
TC-11	FR-08	Admin deletes event	Admin deletes event and confirms with event name	Event and all member rows removed; event gone from all dashboards		

6.4 Validation of Non-Functional Requirements

Each non-functional requirement from Section 3.6 is validated with a specific tool and target, as listed in Table 6.3. Performance and portability are measured at this milestone on the implemented screens; usability will be measured in a user trial once the guest, expense, and task modules are complete, because a meaningful trial needs the full planning workflow.

Table 6.3: Non-Functional Requirement Validation Plan

Requirement	Metric	Target	Method / Tool
Performance	Largest Contentful Paint; action response time (p95)	< 2.5 s on 4G; < 1 s	Lighthouse (mobile, simulated 4G); Vercel analytics
Reliability	Monthly availability; partial writes	>= 99.5%; 0	Vercel / Supabase status; transaction tests
Security	Tables with RLS; rows leaked to non-members	100%; 0	Supabase security advisor; policy test script (Section 6.2.5)
Usability	Time to create event and share invite; SUS score	< 3 min; >= 70	Timed trial and SUS questionnaire with 10+ users
Consistency	Delay before change appears for another member	< 2 s	Two browsers observing the same event
Portability	Supported browsers; usable widths	4 browsers; 360–1920 px	Manual test in Chrome, Edge, Firefox, Safari; responsive mode

6.5 Summary

Festara is tested with black box, unit, integration, and direct database security testing. Eleven test cases cover the completed modules and trace back to fourteen functional requirements, with particular attention to the invite flow and Row Level Security, where a

defect would have the greatest impact. Each non-functional requirement has a measurable target and a defined method of validation. The test results will be recorded in Table 6.2 and summarized in the final report.

Chapter 7

CONCLUSION AND FUTURE WORK

7.1 Introduction

This chapter reviews the progress of Festara against the objectives set in Chapter 1, presents the plan for the remaining work, and states the limitations of the system and directions for future development.

7.2 Progress Review against Objectives

At the 40 percent milestone, the requirements and the design of the complete system are finished, and the three foundation modules on which all other modules depend are implemented and deployed. Table 7.1 compares each objective from Section 1.4 with its current status.

Table 7.1: Progress against Project Objectives

#	Objective	Status
1	Secure registration, login, password reset, and profile management	Completed
2	Create, edit, and delete events with budget	Completed
3	Admin, Member, and Guest roles enforced on every action	Completed (database-enforced through RLS)
4	Guest list, invite links, and RSVP tracking	Invite links completed; guest list and RSVP planned
5	Expense logging with 80 percent budget warning	Designed; implementation planned
6	Task assignment with deadlines and status	Designed; implementation planned
7	Admin dashboard with summary and activity log	Designed; implementation planned
8	AI checklist and budget estimator	Phase 2

7.3 Remaining Work and Timeline

The remaining work is scheduled in two-week sprints, as shown in Figure 7.1. Guest List and RSVP is next, because it extends the invite links already built. Expense and Budget Tracking follows, since it is the feature users asked about most. Tasks and the Admin Dashboard come after that, and the AI features are developed last, followed by a final round of testing, the usability trial, and the final report.

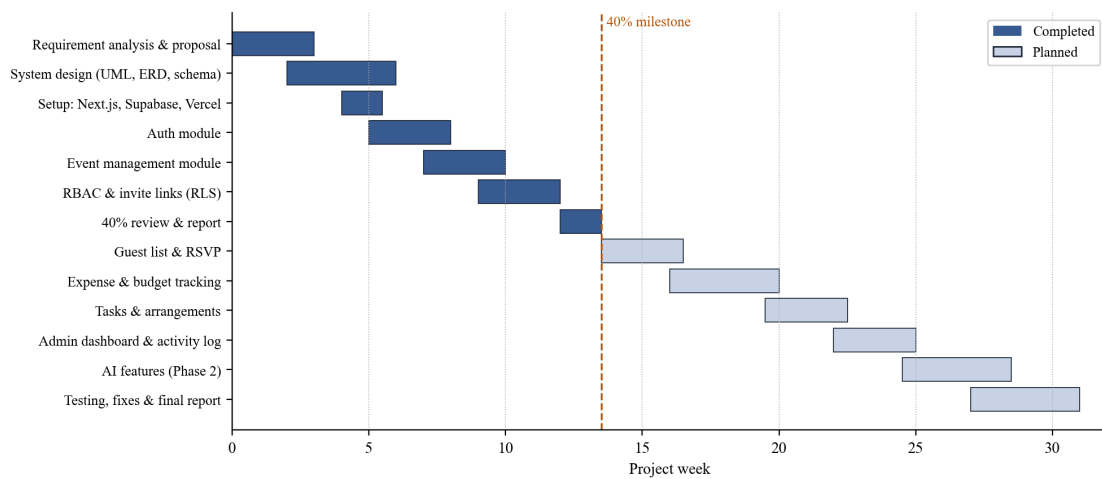


Figure 7.1: Project timeline showing completed and planned work

7.4 Limitations

- Festara records spending but does not transfer money between users; settling up still happens outside the app.
- An active internet connection is required; there is no offline mode in the first release.
- Guest invitations are link-based only; there is no direct SMS or WhatsApp sending in the first phase.
- The system is a web application only; there is no dedicated iOS or Android app in the initial release.
- The AI features depend on the availability of the OpenAI API and carry a per-request cost [3].
- Data is stored on cloud servers, so users with strict data-residency requirements must review the privacy policy before use.

7.5 Future Work

- Integrate WhatsApp Business messaging to send invites and RSVP reminders directly.
- Add a settle-up view that calculates the minimum set of payments between members, similar to Splitwise [10].
- Make the application installable as a Progressive Web App with offline caching of the guest list and task list.
- Add Urdu language support and right-to-left layout for wider use by Pakistani families.

- Offer reusable event templates, such as a standard Mehndi or trip checklist, based on anonymized data from completed events.

7.6 Summary

Festara has reached its 40 percent milestone with a complete specification and design and a deployed foundation of authentication, event management, and database-enforced role-based access. The remaining modules have a clear plan and reuse the patterns already in place. When complete, Festara will give families, friend groups, and student societies a free, shared workspace for planning events together, which the existing tools reviewed in Chapter 2 do not provide.

APPENDIX I: DATA DICTIONARY

This appendix describes the columns of each table in the Festara database. All tables have Row Level Security enabled. Timestamps use the timestamptz type and default to now().

profiles

Column	Type	Constraints	Description
id	uuid	PK, FK → auth.users	Same id as the Supabase Auth user
full_name	text	not null	Display name
phone	text	nullable	Contact number
avatar_url	text	nullable	Profile picture URL
created_at	timestamptz	default now()	Creation time

events

Column	Type	Constraints	Description
id	uuid	PK	Event identifier
name	text	not null	Event name
type	event_type	enum	Wedding, Engagement, Mehndi, Walima, Birthday, Eid Gathering, Trip, University Event, Other
event_date	date	not null	Date of the event
location	text	nullable	Venue or destination
description	text	nullable	Free-text details
total_budget	numeric(12,2)	>= 0	Planned budget in PKR
created_by	uuid	FK → profiles	Creator
created_at, updated_at	timestamptz		Audit times

event_members

Column	Type	Constraints	Description
id	uuid	PK	
event_id	uuid	FK → events, cascade	
user_id	uuid	FK → profiles	
role	member_role	enum: admin, member, guest	Role in this event
joined_at	timestamptz		
(event_id, user_id)		unique	One role per user per event

invitations

Column	Type	Constraints	Description
id	uuid	PK	
event_id	uuid	FK → events, cascade	
token	text	unique, not null	128-bit random URL-safe token
role	member_role	member or guest	Role granted on acceptance
created_by	uuid	FK → profiles	Admin who generated it
expires_at	timestamptz	default now() + 7 days	
used_count	integer	default 0	Times accepted

guests

Column	Type	Constraints	Description
id	uuid	PK	
event_id	uuid	FK → events, cascade	
name	text	not null	
phone	text	nullable	
family_side	text	nullable	e.g. bride's family, groom's family
rsvp_status	rsvp_status	enum, default pending	pending, confirmed, declined
added_by	uuid	FK → profiles	

expenses

Column	Type	Constraints	Description
id	uuid	PK	
event_id	uuid	FK → events, cascade	
paid_by	uuid	FK → profiles	Set from the session
amount	numeric(12,2)	> 0	Amount in PKR
category	expense_category	enum	venue, catering, decor, transport, photography, misc
description	text	nullable	
expense_date	date	not null	

tasks

Column	Type	Constraints	Description
id	uuid	PK	
event_id	uuid	FK → events, cascade	
assigned_to	uuid	FK → profiles, nullable	
title	text	not null	
description	text	nullable	
deadline	date	nullable	
status	task_status	enum, default todo	todo, in_progress, done

activity_log

Column	Type	Constraints	Description
id	bigint	PK, identity	
event_id	uuid	FK → events, cascade	
actor_id	uuid	FK → profiles	Who made the change
action	text	not null	insert, update, delete
entity_type, entity_id	text, uuid		Which record changed
created_at	timestamptz	default now()	

APPENDIX II: USER MANUAL (COMPLETED MODULES)

Registering and Logging In

1. Open the Festara website and select Register.
2. Enter your full name, email, and a password of at least 8 characters, then select Create Account.
3. Open the confirmation email and select the link. You can now log in with your email and password.
4. If you forget your password, select Forgot Password on the login page and follow the emailed link.

Creating an Event

1. On the My Events page, select Create Event.
2. Enter the event name, choose its type, and enter the date, location, description, and total budget.
3. Select Save. You are now the Admin of the event.

Inviting People

1. Open the event and go to the Members tab.
2. Choose the role (Member or Guest) and select Generate Link.
3. Copy the link and share it, for example in your family WhatsApp group. The link works for 7 days.
4. To change someone's role, choose a new role next to their name. To remove them, select Remove and confirm.

REFERENCES

- [1] Supabase Inc., "Supabase Documentation - Database, Auth, and Realtime," 2024. [Online]. Available: <https://supabase.com/docs>. [Accessed: Apr. 2025].
- [2] Vercel Inc., "Next.js Documentation - App Router and Server Components," 2024. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Apr. 2025].
- [3] OpenAI, "OpenAI API Reference - Chat Completions and Structured Outputs," 2024. [Online]. Available: <https://platform.openai.com/docs>. [Accessed: Apr. 2025].
- [4] Vercel Inc., "Vercel AI SDK Documentation," 2024. [Online]. Available: <https://sdk.vercel.ai/docs>. [Accessed: Apr. 2025].
- [5] Associate Events, "The Biggest Pain Points in Event Planning and How to Overcome Them," Aug. 2025. [Online]. Available: <https://www.associate-events.com/post/the-biggest-pain-points-in-event-planning-and-how-to-overcome-them>.
- [6] PYMNTS Intelligence, "Financial Anxiety Spurs Demand for Consumer Budgeting Apps," Jun. 2025. [Online]. Available: <https://www.pymnts.com/personal-finance/2025/financial-anxiety-spurs-demand-for-consumer-budgeting-apps>.
- [7] Seated with Love, "Wedding Venue Software: What It Is and Why Generic Event Tools Do Not Work," Apr. 2025. [Online]. Available: <https://blog.seatedwithlove.com/what-is-wedding-venue-software-and-why-generic-event-tools-dont-work>.
- [8] GetApp, "Best Collaboration Software with Time and Expense Tracking 2025," 2025. [Online]. Available: <https://www.getapp.com/collaboration-software/web-collaboration/f/time-expense-tracking>.
- [9] ChecklistGuro, "The 5 Best Event Planning Management Software of 2025," Dec. 2025. [Online]. Available: <https://checklistguro.com/blog/the-5-best-event-planning-management-software-of-2025>.
- [10] Splitwise Inc., "Splitwise - Split Expenses with Friends," 2024. [Online]. Available: <https://www.splitwise.com>. [Accessed: Apr. 2025].
- [11] S. Zhang, K. Alanezi, M. Gartrell, R. Han, Q. Lv, and S. Mishra, "Understanding group event scheduling via the OutWithFriendz mobile application," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 1, no. 4, 2018. [Online]. Available: <https://arxiv.org/abs/1710.02609>.
- [12] J. Kim, H. Zhang, P. André, L. B. Chilton, W. Mackay, M. Beaudouin-Lafon, R. C. Miller, and S. P. Dow, "Cobi: A community-informed conference scheduling tool," in *Proc. 26th Annu. ACM Symp. User Interface Software and Technology (UIST '13)*, St Andrews, U.K., 2013, pp. 173–182.
- [13] J. Škrabálek, T. Ludík, J. Slabý, and T. Pitner, "Web-based service for collaborative organization of academic events – Case study of 'Takeplace'," in *Proc. 12th Int. Symp. Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, Timisoara, Romania, 2010, doi: 10.1109/SYNASC.2010.66.

- [14] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [15] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001.
- [16] K. Schwaber and J. Sutherland, "The Scrum Guide," Nov. 2020. [Online]. Available: <https://scrumguides.org/scrum-guide.html>.
- [17] The PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. [Online]. Available: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>.